BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

TRAINING CUỐI KỲ HỌC KỲ I NĂM HỌC 2024 – 2025

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

BAN HỌC TẬP

Khoa Công nghệ Phần mềm

Trường Đại học Công nghệ Thông tin

Đại học Quốc gia thành phố Hồ Chí Minh

CONTACT

bht.cnpm.uit@gmail.com

fb.com/bhtcnpm

fb.com/groups/bht.cnpm.uit

TEAM TIẾNG ANH

english.with.bht@gmail.com

creative.owl.se

english.with.bht

TRAINING

HỆ ĐIỀU HÀNH

Thời gian: 19:00 thứ Năm ngày 04/01/2025

Địa điểm: Microsoft Teams – Code: w2dsy1q

Trainers: Nguyễn Quý Phong – KTPM2023.2

Ngô Lê Tấn Huy – MMTT2023.1

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

MỤC LỤC

Chương 5. Đồng bộ tiến trình

Chương 6. Deadlock

Chương 7. Quản lý bộ nhớ

Chương 8. Bộ nhớ ảo

BOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

I. Tổng quan về đồng bộ

II. Các giải pháp đồng bộ

III. Các bài toán đồng bộ

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

I. Tổng quan về đồng bộ

Các thuật ngữ:

Race condition: hiện tượng xảy ra khi các tiến trình cùng truy cập đồng thời vào dữ liệu được chia sẻ. Kết quả cuối cùng sẽ phụ thuộc vào thứ tự thực thi của các tiến trình đang chạy đồng thời với nhau.

Vùng tranh chấp:

Mỗi tiến trình có một **vùng tranh chấp** là một **đoạn code:**

- Thực hiện việc thay đổi giá trị của dữ liệu được chia sẻ (có thể là các biến, bảng dữ liệu, file,...).
- Khi một tiến trình đang thực hiện vùng tranh chấp của mình thì các tiến trình khác KHÔNG được thực hiện vùng tranh chấp của chúng.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

I. Tổng quan về đồng bộ

- Lời giải cho vùng tranh chấp cần phải đảm bảo ba yếu tố:

Mutual exclusion (loại trừ tương hỗ): Khi một tiến trình P đang thực thi trong vùng tranh chấp của nó thì không có tiến trình Q nào khác đang thực thi trong vùng tranh chấp của Q.

Progress (tiến triển): Một tiến trình tạm dừng bên ngoài vùng tranh chấp không được ngăn cản các tiến trình khác vào vùng tranh chấp.

→ Deadlock

Bounded waiting (chờ đợi giới hạn): Mỗi tiến trình chỉ phải chờ để được vào vùng tranh chấp trong một khoảng thời gian có hạn định nào đó.

→ Starvation

CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

Phân loại theo sự hỗ trợ phần cứng:

- Giải pháp phần mềm: các giải thuật.
- Giải pháp phần cứng: Test & Set, Compare & Swap…

Phần loại theo sự hỗ trợ của hệ điều hành:

- Busy waiting: sử dụng kĩ thuật lập trình.
- Sleep & Wakeup: cần hệ điều hành cung cấp cơ chế tạm dừng và đánh thức tiến trình.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

1. Giải pháp Peterson

- Là một giải pháp **phần mềm** dùng **busy waiting**.
- Chỉ dành cho **2 tiến trình**.
- Dùng 2 biến:

  int turn: xác định tiến trình nào đến lượt vào vùng tranh chấp.

  bool flag[2]: xác định liệu tiến trình flag[i] đã sẵn sàng vào CS chưa.
- Đảm bảo mutual exclusion, progress, bounded waiting.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

1. Giải pháp Peterson

flag[i] = true → i sẵn sàng vô CS

turn = j → đổi lượt về lại cho j

Kiểm tra nếu vẫn đang còn lượt của j thì cho nó chạy

Cho process i thực thi trên CS

flag[i] = false → i chưa sẵn sàng vào CS → cho phép j vào vùng CS

i thực hiện remainder section

P_i

```
{
    while (true){
      flag[i] = true;
      turn = j;
      while (flag[j] && turn = = j);

      /* critical section */

      flag[i] = false;
      /* remainder section */
    }
}
```

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

1. Giải pháp Peterson

Trong các kiến trúc hiện đại:

- Với các tiến trình đơn tiểu trình thì việc thực hiện các lệnh sẽ không có gì thay đổi.
- Với các tiến trình đa tiểu trình, việc sắp xếp lại các thao tác có thể dẫn đến kết quả không nhất quán hoặc không dự đoán được.

→ giải pháp Peterson không còn phù hợp với kiến trúc hiện đại.

→ cần sử dụng **Memory Barrier**.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

2. Memory Barrier

- Memory Barrier là một **chỉ thị** (instruction) mà bắt buộc mọi thay đổi trong bộ nhớ phải được truyền tải (hiển thị) đến tất cả bộ xử lý khác.
- Cách hoạt động: đảm bảo là tất cả thao tác load (nạp dữ liệu) và store (ghi dữ liệu) đều đã được hoàn thành trước khi các thao tác load và store sau đó được thực hiện.
- Kể cả khi các câu lệnh được sắp xếp lại thì memory barrier đảm bảo các thao tác ghi dữ liệu vào bộ nhớ đã được hoàn thành rồi và truyền tải đến bộ xử lý khác trước khi thao tác nạp (hoặc ghi) dữ liệu vào bộ nhớ tiếp theo được thực thi.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

3. Mutex locks

- Mutex locks hay còn gọi lại "khóa mutex" là kỹ thuật giúp đảm bảo yêu cầu loại trừ tương hỗ khi các tiến trình thực thi đồng thời với nhau.
- Mutex locks hoạt động như một ổ khóa, khi tiến trình tiến vào vùng tranh chấp thì cần phải yêu cầu khóa ổ khóa lại, tương tự, sau khi ra khỏi vùng tranh chấp thì tiến trình cần yêu cầu mở khóa mutex để tiến trình khác có thể tiến vào.
- Hệ điều hành cung cấp 2 thao tác:

block: tạm dừng và đặt tiến trình gọi thao tác này vào trong hàng đợi – trạng thái ngủ.

wakeup: xóa một tiến trình ra khỏi hàng đợi và đặt lại vào trong hàng đợi sẵn sàng – đánh thức.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

3. Mutex locks

```
while (true) {
    acquire lock
        critical section
    release lock
        remainder section
}
```

```
acquire() {
        if (!available);
                block();
        available = false;
}
```

```
release() {
        available = true;
        wakeup(Q);
}
```

Acquire(): nếu CS đang bận thì block và chờ cho tiến trình khác chạy xong, còn không thì đặt available = false để cho tiến trình này vào CS.

Release(): sau khi thực thi trong CS xong thì gán available = true ám chỉ CS trống và wakeup một tiến trình đang bị block.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

- Tương tự như Mutex, Semaphore là công cụ để các tiến trình có thể đồng bộ các hoạt động/ hành vi của mình.
- Semaphore S về bản chất là một **biến số nguyên.**
- Chỉ có thể được truy cập thông qua 2 thao tác: wait() và signal () – hay còn được gọi là P() và V().

WAIT	SIGNAL
Nếu semaphore S không dương thì tiến trình/ tiểu trình phải vào hàng đợi. Khi tiến trình thực thi CS thì giá trị S giảm đi 1. Được sử dụng khi muốn sử dụng tài nguyên.	Tăng giá trị S lên 1 đơn vị. Được sử dụng khi muốn trả lại tài nguyên.

Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

Semaphore có 2 loại: counting semaphore và binary semaphore.

Hiện thực semaphore **không busy waiting**:

Mỗi semaphore được gắn với một hàng đợi (FIFO).

Mỗi phần tử trong hàng đợi có 2 phần:

- Giá trị số nguyên (giá trị của semaphore).
- Con trỏ chỉ đến phần tử tiếp theo (danh sách liên kết đơn).

Khi S->value ≥ 0: số lần mà các tiến trình/tiểu trình có thể thực thi wait(S) mà không bị blocked là S->value.

Khi S->value < 0: số tiến trình/ tiểu trình đang đợi trên S là |S->value|.

Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

Định nghĩa cấu trúc semaphore

```
typedef struct {
        int value;
        struct process *list;
    } semaphore;
```

Định nghĩa hàm wait và signal

```
Wait(semaphore* S) {
        S->value--;
        if (S->value < 0) {
            add this process to S->list;
            block();
        }
}
```

```
signal(semaphore* S) {
        S->value++;
        if (S->value >= 0) {
            remove a process P from S->list;
            wakeup(P);
        }
}
```

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

Ví dụ 1: Đồng bộ hóa P1, P2 để P1 thực thi S1 trước S2, S2 trước S3, với S1 và S3 là 2 thao tác trong P1, S2 là thao tác trong P2.

Ý tưởng: dùng semS1, semS2 lần lượt đại diện cho thời điểm S2, S3 được chạy.

Khởi tạo: semS1 = 0, semS2 = 0.

```
P1:
S1
signal(semS1)
wait(semS2)
S3
```

```
P2:
wait(semS1)
S2
Signal(semS2)
```

Đảm bảo thứ tự thực thi

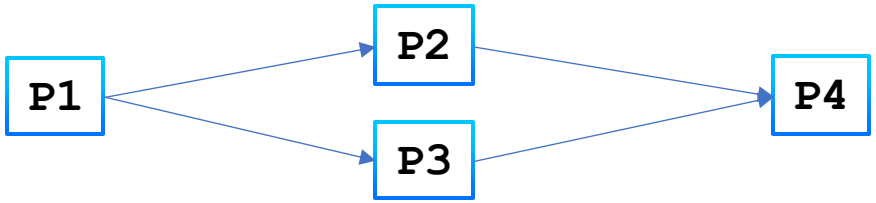
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

<u>Ví dụ 2:</u> Đồng bộ hóa P1, P2, P3, P4 để P1 chạy thì P2, P3 được chạy, sau khi cả P2 và P3 chạy thì P4 mới được chạy.



Ý tưởng:

- P4 được chạy khi cả P2 và P3 hoàn tất → tiến trình ấy cần gọi wait() 2 lần với mục đích xem thử giá trị semaphore có >= 2 hay không.
- P2 và P3 chạy được khi P1 hoàn tất → P1 cần gọi signal() 2 lần để cho cả P2 và P3 đều có thể tiến vào vùng tranh chấp (không quan tâm thứ tự).

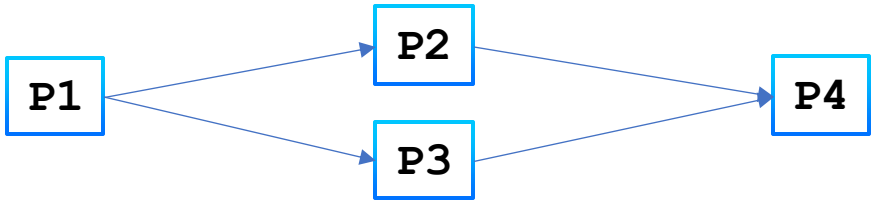
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

<u>Ví dụ 2:</u> Đồng bộ hóa P1, P2, P3, P4 để P1 chạy thì P2, P3 được chạy, sau khi cả P2 và P3 chạy thì P4 mới được chạy.



Mutual exclusion??

Khởi tạo: sem1 = 0, sem4 = 0.

```
P1:
Thực thi
signal(sem1)
signal(sem1)
```

```
P2:
wait(sem1)
Thực thi
signal(sem4)
```

```
P3:
wait(sem1)
Thực thi
signal(sem4)
```

```
P4:
wait(sem4)
wait(sem4)
Thực thi
```

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

4. Semaphore

Đoạn mã định nghĩa các lệnh wait(S) và signal(S) cũng chính là vùng tranh chấp.

Giải pháp cho vùng tranh chấp wait(S) và signal(S):

- Uniprocessor: có thể dùng cơ chế cấm ngắt (disable interrupt). Nhưng phương pháp này không làm việc trên hệ thống multiprocessor.

- Multiprocessor: có thể dùng các giải pháp software (như giải thuật Dekker, Peterson) hoặc giải pháp hardware (TestAndSet, Swap) → đều là busy waiting nhưng được sử dụng vì số câu lệnh trong vùng tranh chấp này thường rất ít.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

5. Monitor

Khái niệm:

- Là một cấu trúc **ngôn ngữ bậc cao**, có chức năng như semaphore nhưng dễ điều khiển hơn.
- Có thể hiện thực bằng semaphore.

Cấu tạo: là một **module phần mềm**, bao gồm:

- Một hoặc nhiều thủ tục (procedure).
- Một đoạn code khởi tạo (initialization code).
- Các biến dữ liệu cục bộ (local data variable).

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

II. Các giải pháp đồng bộ

5. Monitor

```
monitor monitor-name
{
        // shared variable declarations
        procedure P1 (…) { …. }
        procedure P2 (…) { …. }
        procedure Pn (…) {……}
        initialization code (…) { … }
}
```

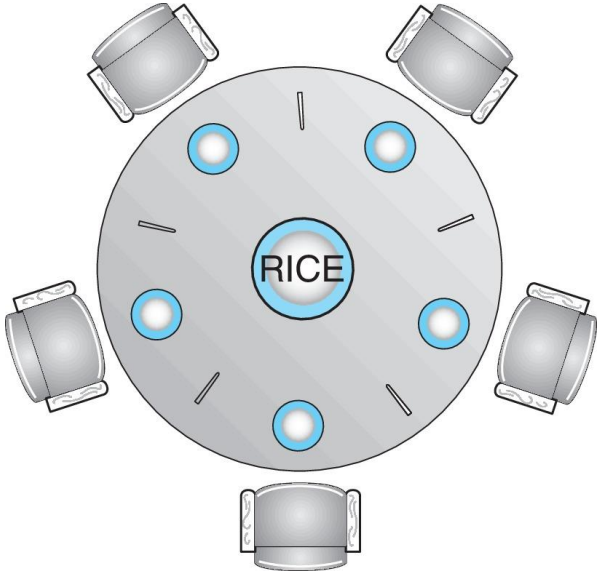
- Tiến trình "vào monitor" bằng cách gọi một trong các thủ tục được định nghĩa trong monitor.
- Chỉ có một tiến trình có thể vào monitor tại một thời điểm → mutual exclusion được bảo đảm.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

III. Các bài toán đồng bộ

1. Bài toán Dining-Philosopher



Đề bài:

Có n Triết gia ngồi trên một chiếc bàn tròn với tô cơm ở giữa và n chiếc đũa. Họ dành thời gian chỉ để làm 2 công việc: suy nghĩ và ăn. Cần phải có đủ 2 chiếc đũa thì mới ăn được, sau khi ăn xong thì trả lại 2 chiếc đũa.

→ Deadlock xảy ra khi mỗi triết gia giữ 1 chiếc đũa.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

III. Các bài toán đồng bộ

1. Bài toán Dining-Philosopher

Một số giải pháp có thể tránh deadlock:

- Cho phép tối đa 4 Triết gia ngồi vào bàn.
- Chỉ cho phép Triết gia cầm đũa khi cả 2 chiếc đũa đã sẵn sàng.

→ Cần phải thực hiện hành động cầm đũa trong CS.

- Giải pháp bất đối xứng: Triết gia ngồi vị trí lẻ cầm đũa bên trái trước, rồi sau đó cầm đũa bên phải; trong khi Triết gia ngồi vị trí chẵn cầm đũa bên phải trước, rồi sau đó cầm đũa bên trái.

Cần phải lưu ý tình trạng starvation vẫn có thể xảy ra.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

III. Các bài toán đồng bộ

2. Bài toán Bounded-buffer

Có mảng gồm n phần tử, 2 tiến trình Producer và Consumer:

- Tiến trình Producer thêm phần tử vào mảng.
- Tiến trình Consumer liên tục lấy phần tử ra khỏi mảng.
- Đảm bảo Producer không thêm phần tử vào khi mảng đã đầy và Consumer không lấy phần tử khi mảng rỗng.

Giải pháp:

- Dùng semaphore full để thể hiện số vị trí có thể xóa (khởi tạo 0), empty để thể hiện số vị trí còn để thêm vào (khởi tạo n), và mutex để bảo vệ 2 vùng tranh chấp.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

III. Các bài toán đồng bộ

3. Bài toán Readers and Writers

Dữ liệu được chia sẻ giữa các tiến trình đang thực thi đồng thời:

- Readers: Chỉ đọc dữ liệu; **không** thực hiện cập nhật.
- Writers: Có thể **vừa đọc vừa ghi** dữ liệu.

Vấn đề:

- Cho phép **nhiều Readers** cùng đọc dữ liệu đồng thời.
- Chỉ **một Writers** được phép truy cập dữ liệu tại một thời điểm.

Giải pháp: có 2 giải pháp là ưu tiên Readers và ưu tiên Writers, nhưng slide chỉ đề cập đến ưu tiên Readers.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

III. Các bài toán đồng bộ

3. Bài toán Readers and Writers

- Dùng semaphore **rw_mutex** để bảo vệ cơ sở dữ liệu được chia sẻ và **mutex** để bảo vệ biến read_count (số lượng readers).

- Nếu một Writer đang ở trong CS và có n Readers đang đợi thì **một Reader** được xếp trong hàng đợi của **rw_mutex** và **n – 1 Reader** kia trong hàng đợi của **mutex**.

- Khi Writer thực thi **signal(rw_mutex),** hệ thống có thể phục hồi thực thi của một trong các Reader đang đợi hoặc Writer đang đợi.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

CÂU HỎI ÔN TẬP

1. Lựa chọn nào sau đây là một giải pháp cho bài toán Dining-Philosopher?

A. Cho phép tối đa 5 triết gia ngồi vào bàn ăn.

B. Các triết gia đều phải cầm đũa bên trái trước.

C. Chỉ cho phép triết gia cầm đũa khi hai đũa sẵn sàng.

D. Các triết gia chỉ được ăn khi đã cầm hai đũa.

➔ Đáp án: C. Chỉ cho phép triết gia cầm đũa khi hai đũa sẵn sàng.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

CÂU HỎI ÔN TẬP

2. Vấn đề nào xảy ra nếu không có sự kiểm soát các dữ liệu được chia sẻ?

A. Tất cả các tiến trình đều có thể truy xuất được dữ liệu.

B. Tiến trình bị ngừng hoạt động.

C. Dữ liệu không nhất quán.

D. Không có vấn đề gì xảy ra.

➔ **Đáp án: C. Dữ liệu không nhất quán.**

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

CÂU HỎI ÔN TẬP

3. Chọn phát biểu SAI về semaphore?

A. Thao tác signal() sẽ khôi phục tiến trình đã bị chặn bởi lệnh wait(S) trước đó nếu giá trị của S âm.

B. Nếu giá trị của semaphore bằng -3 thì có 3 tiến trình đang bị blocked trong hang đợi của S.

C. Semaphore là giải pháp phần mềm cần sự trợ giúp của hệ điều hành.

D. Tiến trình gọi lệnh wait(S) sẽ bị chặn nếu giá trị của S dương.

➔ Đáp án: D. Tiến trình gọi lệnh wait(S) sẽ bị chặn nếu giá trị của S dương.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

CÂU HỎI ÔN TẬP

4. Để cho 4 tiến trình cùng vào vùng tranh chấp, cần khởi tạo semaphore bằng mấy?

A. 1 B. 3 C. 4 D. 5

➔ Đáp án: C. 4

5. Có bao nhiêu Semaphore được dùng trong bài toán Bounded-buffer?

A. 1 B. 3 C. 4 D. 5

➔ Đáp án: B. 3

6. Cơ chế đồng bộ nào sau đây có hiệu dụng CPU tốt nhất:

A. Giải thuật Bakery. B. Test & Set.

C. Giải thuật Peterson. D. Mutex.

➔ Đáp án: D. Mutex

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 5. ĐỒNG BỘ TIẾN TRÌNH

CÂU HỎI ÔN TẬP

7. Nối những giải pháp đồng bộ sau với những đặc điểm thích hợp?

1. Memory barrier
2. Monitor
3. Mutex
4. Semaphore

a. Về bản chất là một số nguyên

b. Có 2 thao tác acquire và release

c. Là một module phần mềm bao gồm: procedure, initialization code và local data variable

d. Là một chỉ thị bắt buộc mọi thay đổi trong bộ nhớ phải truyền đến bộ xử lý khác

BAN HỌC TẬP
Sharing is learning

Chương 6

DEADLOCK

BOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6: DEADLOCK

I. Vấn đề deadlock

II. Các tính chất của deadlock

III. Phương pháp giải quyết deadlock

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

I. Vấn đề deadlock

6.1. Định nghĩa

– Một tiến trình gọi là deadlock nếu nó đang **đợi** một sự kiện mà sẽ **không bao giờ** xảy ra.

- Thường sẽ có nhiều hơn một tiến trình bị liên quan trong một deadlock.

– Một tiến trình được gọi là **trì hoãn vô hạn định** nếu nó bị trì hoãn một khoảng thời gian dài lặp đi lặp lại trong khi hệ thống đáp ứng cho những tiến trình khác.

- **Ví dụ:** Một tiến trình sẵn sàng để xử lý nhưng nó không bao giờ nhận được CPU.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

II. Các tính chất của deadlock

6.2. Điều kiện xảy ra deadlock

– Có 4 tình huống dẫn đến deadlock:

+ **Mutual exclution**: trong hệ thống có ít nhất một tài nguyên được giữ theo nonsharable mode.

+ **Hold and wait**: một tiến trình đang giữ ít nhất một tài nguyên và đợi thêm tài nguyên do quá trình khác giữ.

+ **No preemption**: tài nguyên không thể bị lấy lại mà chỉ có thể được trả lại từ tiến trình đang giữ tài nguyên đó khi nó muốn.

+ **Circular wait**: một tập hợp các process đang chờ đợi lẫn nhau ở dạng vòng tròn (chu trình)

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

II. Các tính chất của deadlock

6.3. Đồ thị cấp phát tài nguyên RAG

– Là đồ thị có hướng, với tập đỉnh V và tập cạnh E.

– Tập đỉnh V gồm 2 loại:

+ P = {P_1, P_2,…,P_n} (All process)

+ R = {R_1, R_2,…,R_n} (All resource)

- Tập cạnh E gồm 2 loại:

+ Cạnh yêu cầu: $P_i \rightarrow R_j$

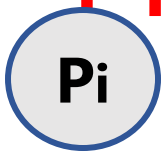
+ Cạnh cấp phát: $R_j \rightarrow P_i$

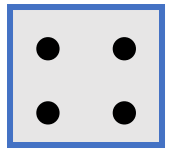
BAN HỌC TẬP
Sharing is learning

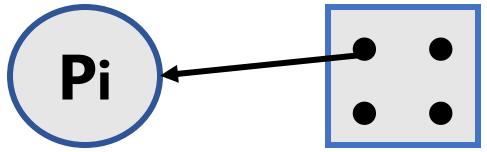
CHƯƠNG 6. DEADLOCK

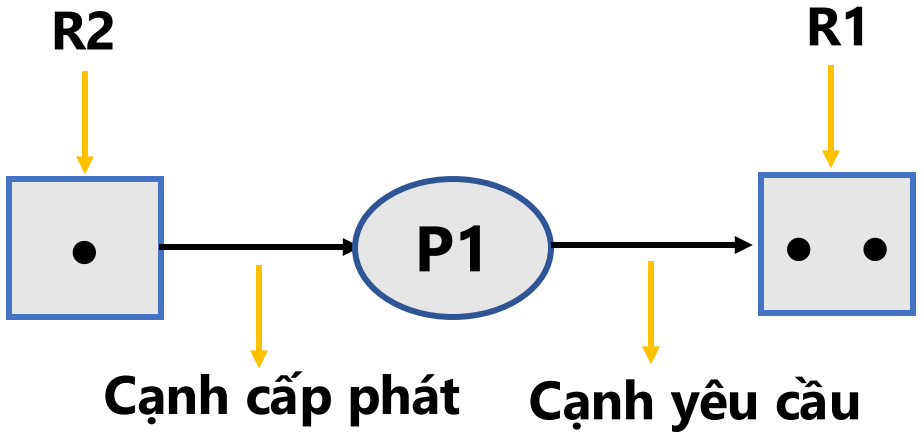
II. Các tính chất của deadlock

6.3. Đồ thị cấp phát tài nguyên RAG

- Process i 

- Loại tài nguyên Rj với 4 thực thể 
- Pi yêu cầu 1 thực thể của Rj

- Pi đang giữ 1 thực thể của Rj 



CHƯƠNG 6. DEADLOCK

II. Các tính chất của deadlock

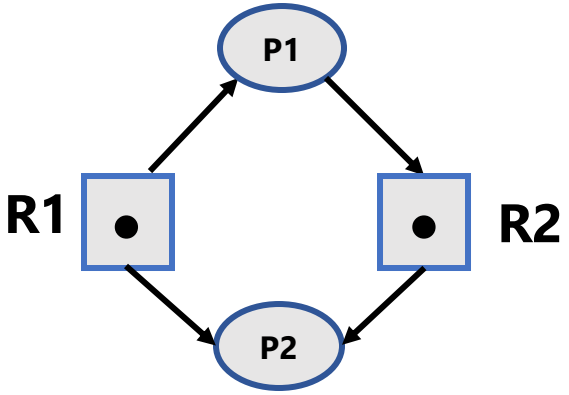
6.4. Mối quan hệ giữa RAG và deadlock

Nếu RAG không chứa chu trình ⇒ Không có deadlock

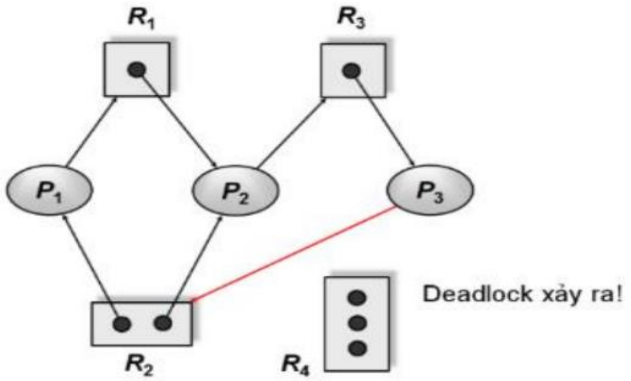
- Nếu RAG chứa một (hay nhiều) chu trình:

+ Nếu mỗi loại tài nguyên chỉ có một thực thể ⇒ có deadlock (1)

+ Nếu mỗi loại tài nguyên có nhiều thực thể ⇒ có thể có deadlock (2)



(1) Một thực thể



(2) Nhiều thực thể

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.5. Tổng quan các phương pháp

– Các phương pháp giải quyết deadlock:

+ **Ngăn deadlock**: không cho phép (ít nhất) một trong 4 điều kiện cần cho deadlock.

+ **Tránh deadlock**: các quá trình cần cung cấp thông tin về tài nguyên nó cần để hệ thống cấp phát tài nguyên một cách thích hợp.

+ Cho phép hệ thống vào trạng thái deadlock, nhưng sau đó phát hiện deadlock và phục hồi hệ thống.

+ Xem như deadlock không bao giờ xảy ra trong hệ thống.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.6. Ngăn deadlock

– Ngăn deadlock bằng cách ngăn một trong 4 điều kiện cần của deadlock:

+ **Ngăn mutual exclusion** (loại trừ tương hỗ):

- Đối với tài nguyên không chia sẻ (printer): không làm được.
- Đối với tài nguyên chia sẻ (read-only file): không cần thiết.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.6. Ngăn deadlock

- **Ngăn hold and wait** (giữ và đợi thêm tài nguyên):

  - Mỗi tiến trình sẽ yêu cầu toàn bộ tài nguyên cần thiết trong một lần. Nếu có đủ tài nguyên thì hệ thống sẽ cấp phát, nếu không đủ tài nguyên thì tiến trình phải bị block.
  - Khi yêu cầu tài nguyên, tiến trình không được giữ tài nguyên nào. Nếu đang có giữ tài nguyên thì phải trả lại trước khi yêu cầu.

Lưu ý: Không thể áp dụng trong thực tế vì process không thể xác định được tài nguyên cần thiết trước khi yêu cầu, và process có thể giữ tài nguyên trong một thời gian dài, chưa thể trả ngay.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.6. Ngăn deadlock

- **Ngăn no preemption**: tiến trình A giữ tài nguyên và đang yêu cầu tài nguyên khác nhưng tài nguyên này chưa được cấp phát ngay:

• Hệ thống lấy lại mọi tài nguyên tiến trình A đang giữ

• Hệ thống sẽ xem xét tài nguyên tiến trình A yêu cầu

- Được giữ bởi một tiến trình khác đang đợi thêm tài nguyên

→ Hệ thống lấy lại và cấp cho A

- Được giữ bởi tiến trình không đợi tài nguyên, A phải đợi

→ Tài nguyên của A bị lấy

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.6. Ngăn deadlock

- **Ngăn circular wait**(chu trình đợi):

● Gán số thứ tự cho tất cả các tài nguyên trong hệ thống. Một process không thể yêu cầu một tài nguyên ít ưu tiên hơn.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.7. Tránh deadlock

- Ngăn deadlock sử dụng tài nguyên không hiệu quả
- Tránh deadlock vẫn đảm bảo hiệu suất sử dụng tài nguyên tối đa đến mức có thể.
- Yêu cầu mỗi tiến trình khai báo số lương tài nguyên tối đa cần để thực hiện cộng việc.
- Giải thuật deadlock sẽ kiểm tra trạng thái cấp phát tài nguyên để đảm bảo hệ thống không rơi vào deadlock.
- Trạng thái cấp phát tài nguyên được định nghĩa dựa trên số tài nguyên còn lại, số tài nguyên đã được cấp phát và yêu cầu tối đa của các tiến trình

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.7. Tránh deadlock

- Trạng thái **an toàn (safe)** và **không an toàn (unsafe)**:

+ Một trạng thái của hệ thống được gọi là an toàn (safe) nếu tồn tại một chuỗi thứ tự an toàn.

+ Một chuỗi quá trình <P1,P2,…,Pn> là một chuỗi an toàn nếu:

● Với mọi i=1,2,…,n yêu cầu tối đa về tài nguyên của Pi có thể được thỏa bởi các tài nguyên mà hệ thống đang có sẵn sàng, cùng với các tài nguyên mà tất cả các Pj (j<i) đang giữ.

+ Một trạng thái của hệ thống được gọi là không an toàn (unsafe) nếu không tồn tại một chuỗi an toàn.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.7. Tránh deadlock

- Ví dụ: hệ thống có 18 files và 3 tiến trình P0, P1, P2, hãy cho biết hệ thống sau có an toàn hay không?

	Cần tối đa	Đang giữ	Cần thêm
P0	11	4	7
P1	12	7	5
P2	6	4	2

- Ta có chuỗi <P2,P0,P1> là chuỗi an toàn ⇒ hệ thống an toàn

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.7. Tránh deadlock

- Nếu hệ thống đang ở trạng thái an toàn (safe) thì không có deadlock xảy ra.
- Nếu hệ thống đang ở trạng thái không an toàn (unsafe) thì có thể dẫn đến deadlock.

⇒ Tránh deadlock bằng cách đảm bảo hệ thống không đi đến trạng thái unsafe.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.8. Các giải thuật tránh deadlock

- Đối với mỗi tài nguyên chỉ có một thực thể thì sử dụng giải thuật đồ thị cấp phát tài nguyên
- Đối với mỗi tài nguyên có nhiều thực thể thì sử dụng giải thuật Banker.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật Banker

- Điều kiện:

+ Mỗi tiến trình phải khai báo số lượng thực thể tối đa của mỗi loại tài nguyên mà nó cần.

+ Khi tiến trình yêu cầu tài nguyên thì có thể phải đợi.

+ Khi tiến trình đã có được đầy đủ tài nguyên thì phải hoàn trả trong một khoảng thời gian hữu hạn nào đó.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật Banker

Bước 1: Tìm Need = Max – Allocation

Bước 2: Tìm tiến trình Pi thỏa:

+ Pi chưa hoàn thành thực thi

+ Needi ≤ Available

+ Nếu không còn Pi thỏa → chuyển sang bước 4

Bước 3: Available = Available + Allocationi

Thêm Pi vào chuỗi

Chuyển sang bước 2

Bước 4: Nếu chuỗi có tồn tại đủ hết các process

⇒ Hệ thống tồn tại chuỗi an toàn

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật Banker

Ví dụ:

	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3
P1	2	0	0	3	2	2				1	2	2
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

Available	Chuỗi an toàn
3 3 2	
5 3 2	P1
7 4 3	P3
7 4 5	P4
10 4 7	P2
10 5 7	P0

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật Banker (cấp phát tài nguyên)

Bước 1: Nếu requesti ≤ needi thì đến bước 2. Nếu không, báo lỗi vì tiến trình đã vượt yêu cầu tối đa.

Bước 2: Nếu requesti ≤ available thì qua bước 3. Nếu không, Pi phải chờ vì tài nguyên không còn đủ để cấp phát.

Bước 3: Giả định cấp phát tài nguyên đáp ứng yêu cầu của Pi bằng cách cập nhật trạng thái hệ thống như sau:

- available = available – requesti
- allocationi = allocationi + requesti
- needi = needi – requesti

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật banker

Ví dụ (trích đề thi HDH HKI năm 2023 – 2024):

Tiến trình	Allocation				Max			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	2	2	1	3	2	7	4	3
P2	1	1	2	1	7	2	4	2
P3	2	5	4	1	8	6	5	7
P4	2	1	2	2	3	2	2	4
P5	0	1	2	4	1	5	7	4

Available			
R1	R2	R3	R4
2	4	2	3

Nếu tiến trình P5 yêu cầu thêm tài nguyên (1,3,2,0), hệ thống có đáp ứng không ,giải thích vì sao?

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật banker

Ví dụ (trích đề thi HDH HKI năm 2023 – 2024):

Available			
R1	R2	R3	R4
2	4	2	3

Tiến trình	Allocation				Max			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	2	2	1	3	2	7	4	3
P2	1	1	2	1	7	2	4	2
P3	2	5	4	1	8	6	5	7
P4	2	1	2	2	3	2	2	4
P5	0	1	2	4	1	5	7	4

Need			
R1	R2	R3	R4
0	5	3	0
6	1	2	1
6	1	1	6
1	1	0	2
1	4	5	0

Nếu tiến trình P5 yêu cầu thêm tài nguyên (1,3,2,0), hệ thống có đáp ứng không ,giải thích vì sao?

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật banker

Ví dụ (trích đề thi HDH HKI năm 2023 – 2024):

Available			
R1	R2	R3	R4
1	1	0	3

Tiến trình	Allocation				Max			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	2	2	1	3	2	7	4	3
P2	1	1	2	1	7	2	4	2
P3	2	5	4	1	8	6	5	7
P4	2	1	2	2	3	2	2	4
P5	1	4	4	4	1	5	7	4

Need			
R1	R2	R3	R4
0	5	3	0
6	1	2	1
6	1	1	6
1	1	0	2
0	1	3	0

Nếu tiến trình P5 yêu cầu thêm tài nguyên (1,3,2,0), hệ thống có đáp ứng không ,giải thích vì sao?

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.9. Giải thuật banker

Ví dụ (trích đề thi HDH HKI năm 2023 – 2024):

Tiến trình	Allocation				Max				Need				Available	Chuỗi an toàn
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	1 1 0 3	
P1	2	2	1	3	2	7	4	3	0	5	3	0	3 2 2 5	P4
P2	1	1	2	1	7	2	4	2	6	1	2	1	4 6 6 9	P5
P3	2	5	4	1	8	6	5	7	6	1	1	6	6 8 7 12	P1
P4	2	1	2	2	3	2	2	4	1	1	0	2	7 9 9 13	P2
P5	1	4	4	4	1	5	7	4	0	1	3	0	9 14 13 14	P3

- Trạng thái mới là safe (chuỗi an toàn là <P4,P5,P1,P2,P3>) vậy có thể cấp phát tài nguyên cho P5

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.10. Phát hiện deadlock

- Đối với mỗi loại tài nguyên chỉ có một thực thể

⇒ Sử dụng wait – for graph

- Đối với mỗi loại tài nguyên có nhiều thực thể:

+ Thực hiện tương tự giải thuật Banker

+ Thay cột Need bằng cột Request

+ Liệt kê chuỗi an toàn ⇒ Tiến trình không có trong chuỗi an toàn thì deadlock xảy ra tại tiến trình đó.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

III. Phương pháp giải quyết deadlock

6.11. Phục hồi deadlock

- Khi deadlock xảy ra, để phục hồi:

+ Báo người vận hành.

+ Hệ thống tự phục hồi bằng cách bẻ gãy chu trình deadlock:

- Chấm dứt một hay nhiều tiến trình
- Lấy lại tài nguyên từ một hay nhiều tiến trình

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 1 (CK2 năm 2023-2024): Lựa chọn nào dưới đây không phải là điều kiện cần để xảy ra deadlock:

A. Một tiến trình đang giữ ít nhất một tài nguyên và đợi thêm tài nguyên do tiến trình khác giữ.

B. Tài nguyên không thể bị lấy lại mà chỉ có thể được trả lại từ tiến trình đang giữ tài nguyên đó khi nó muốn.

C. Ít nhất một tài nguyên được giữ theo chế độ không chia sẻ.

D. Mỗi tiến trình chỉ có thể yêu cầu thực thể của một loại tài nguyên theo thứ tự tăng dần của loại tài nguyên.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 1 (CK2 năm 2023-2024): Lựa chọn nào dưới đây không phải là điều kiện cần để xảy ra deadlock:

A. Một tiến trình đang giữ ít nhất một tài nguyên và đợi thêm tài nguyên do tiến trình khác giữ.

B. Tài nguyên không thể bị lấy lại mà chỉ có thể được trả lại từ tiến trình đang giữ tài nguyên đó khi nó muốn.

C. Ít nhất một tài nguyên được giữ theo chế độ không chia sẻ.

D. Mỗi tiến trình chỉ có thể yêu cầu thực thể của một loại tài nguyên theo thứ tự tăng dần của loại tài nguyên.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 2 (CK1 năm 2023-2024): Khi deadlock xảy ra, nếu muốn phục hồi bằng cách chấm dứt tiến trình, cần thực hiện việc này như thế nào sao cho hiệu quả?

A. Chấm dứt toàn bộ tiến trình deadlock.

B. Chấm dứt lần lượt từng tiến trình cho đến khi không còn deadlock.

C. Chấm dứt các tiến trình không bị deadlock.

D. Chọn ngẫu nhiên một tiến trình bị deadlock và chỉ chấm dứt tiến trình này.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 2 (CK1 năm 2023-2024): Khi deadlock xảy ra, nếu muốn phục hồi bằng cách chấm dứt tiến trình, cần thực hiện việc này như thế nào sao cho hiệu quả?

A. Chấm dứt toàn bộ tiến trình deadlock.

B. Chấm dứt lần lượt từng tiến trình cho đến khi không còn deadlock.

C. Chấm dứt các tiến trình không bị deadlock.

D. Chọn ngẫu nhiên một tiến trình bị deadlock và chỉ chấm dứt tiến trình này.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 3 (CK2 năm 2023-2024): Phát biểu sau đây là ĐÚNG hay SAI?

Phát biểu: Giải thuật ngăn deadlock sẽ kiểm tra trạng thái cấp phát tài nguyên để đảm bảo hệ thống không rơi vào deadlock?

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 3 (CK2 năm 2023-2024): Phát biểu sau đây là ĐÚNG hay SAI?

Phát biểu: Giải thuật ngăn deadlock sẽ kiểm tra trạng thái cấp phát tài nguyên để đảm bảo hệ thống không rơi vào deadlock?

Đáp án: SAI

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 4 (CK2 năm 2023-2024): Trả lời ngắn gọn câu hỏi sau bằng tiếng Anh

Để phát hiện deadlock trong trường hợp mỗi loại tài nguyên chỉ có một thực thể, cần sử dụng loại đồ thị nào, biết rằng nó tạo ra từ đồ thị cấp phát tài nguyên?

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 6. DEADLOCK

Một số câu hỏi về deadlock.

Câu 4 (CK2 năm 2023-2024): Trả lời ngắn gọn câu hỏi sau bằng tiếng Anh

Để phát hiện deadlock trong trường hợp mỗi loại tài nguyên chỉ có một thực thể, cần sử dụng loại đồ thị nào, biết rằng nó tạo ra từ đồ thị cấp phát tài nguyên?

Đáp án: wait-for graph

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

I. Khái niệm cơ sở

II. Các kiểu địa chỉ nhớ

III. Chuyển đổi địa chỉ nhớ

IV. Mô hình quản lý bộ nhớ

V. Cơ chế phân trang

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

I. Khái niệm cơ sở

- Quản lý bộ nhớ là công việc của hệ điều hành với sự hỗ trợ của phần cứng nhằm phân phối, sắp xếp các tiến trình trong bộ nhớ sao cho hiệu quả.
- Mục tiêu: là nạp càng nhiều tiến trình vào bộ nhớ càng tốt (gia tăng mức độ đa chương).
- Trong hầu hết các hệ thống, **kernel sẽ chiếm một phần cố định của bộ nhớ,** phần còn lại phân phối cho các tiến trình.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

I. Khái niệm cơ sở

Các yêu cầu đối với quản lý bộ nhớ:

- Cấp phát bộ nhớ cho các tiến trình.
- Tái định vị (relocation): khi swapping…
- Bảo vệ: kiểm tra truy xuất bộ nhớ có hợp lệ không.
- Chia sẻ: cho phép các tiến trình chia sẻ vùng nhớ chung.
- Kết gán địa chỉ nhớ luận lý của user vào địa chỉ thực.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

II. Các kiểu địa chỉ nhớ

Địa chỉ vật lý (physical address) (hay là địa chỉ thực) là một vị trí thực trong bộ nhớ chính.

Địa chỉ luận lý (logical address) là một vị trí nhớ được diễn tả trong một chương trình (còn gọi là địa chỉ ảo - virtual address):

- Địa chỉ tuyệt đối (absolute address): địa chỉ tương đương với địa chỉ thực.
- Địa chỉ tương đối (relative address) (địa chỉ khả tái định vị, relocatable address) là một kiểu địa chỉ luận lý trong đó các địa chỉ được biểu diễn tương đối so với một vị trí xác định nào đó trong chương trình.

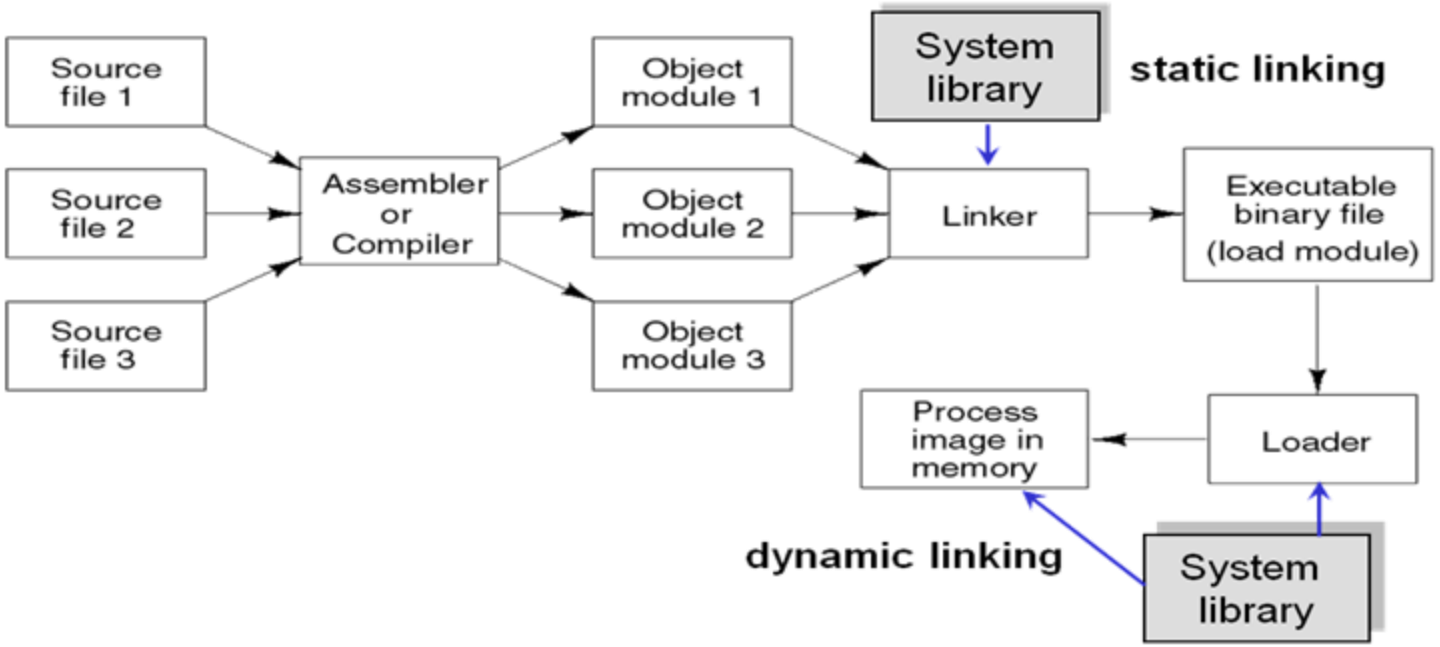
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

II. Các kiểu địa chỉ nhớ

Bộ linker: kết hợp các object module thành một file nhị phân khả thực thi gọi là load module.

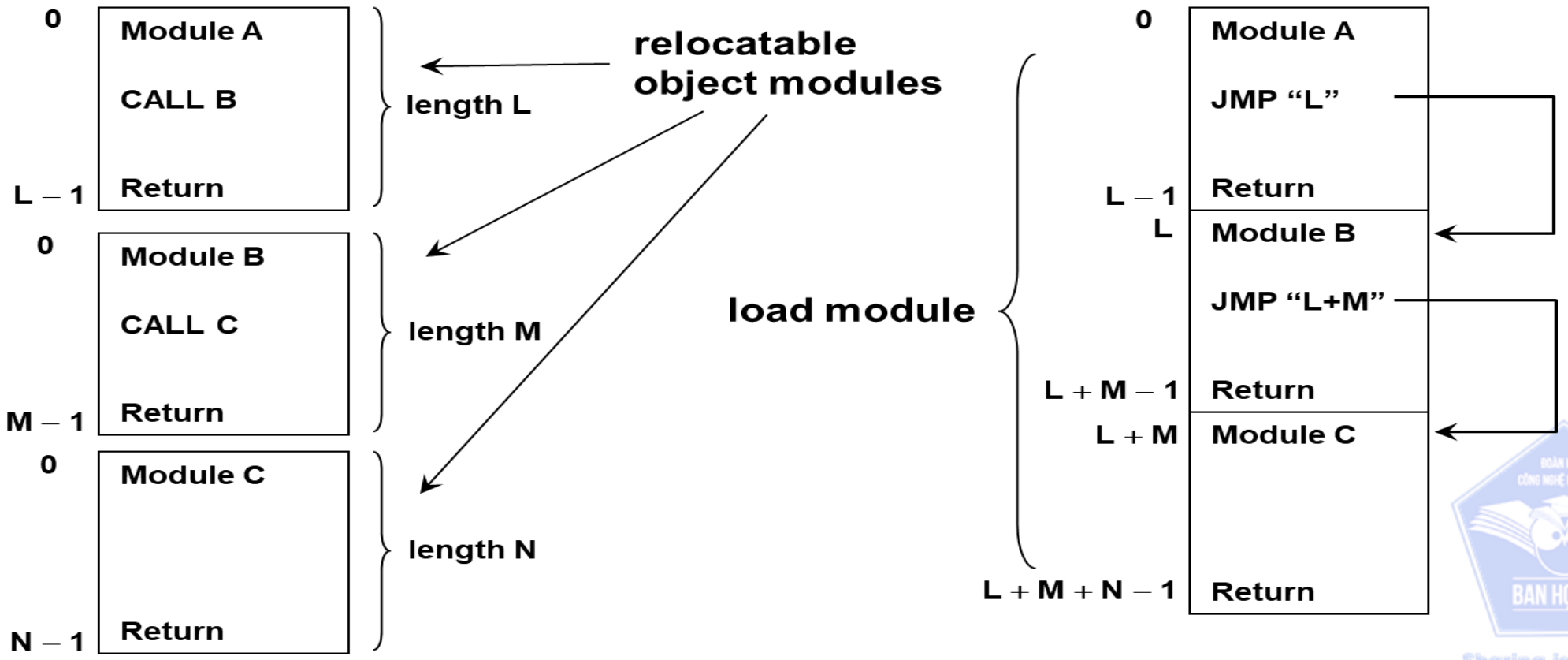
Bộ loader: nạp (load) module vào bộ nhớ chính.



CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

II. Các kiểu địa chỉ nhớ

Cơ chế thực hiện linking:



CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

III. Chuyển đổi địa chỉ

Chuyển đổi địa chỉ: quá trình ánh xạ một địa chỉ từ không gian địa chỉ này sang không gian địa chỉ khác.

Địa chỉ lệnh và dữ liệu được chuyển đổi thành địa chỉ thực có thể xảy ra tại **ba thời điểm** khác nhau:

- Compile time: nếu biết trước địa chỉ bộ nhớ của chương trình thì có thể kết gán địa chỉ tuyệt đối lúc biên dịch.
    - Ví dụ: chương trình .COM của MS. DOC.
    - Khuyết điểm: phải biên dịch lại nếu thay đổi địa chỉ nạp chương trình.
- Loading time: vào thời điểm loading, loader phải chuyển đổi địa chỉ khả tái định vị thành địa chỉ thực dựa trên một địa chỉ nền.
    - Địa chỉ thực được tính toán vào thời điểm nạp chương trình.
    - Phải tiến hành reload nếu địa chỉ nền thay đổi.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

III. Chuyển đổi địa chỉ

- Execution time: khi trong quá trình thực thi, tiến trình có thể được di chuyển từ segment này sang segment khác trong bộ nhớ thì quá trình chuyển đổi địa chỉ được trì hoãn đến thời điểm thực thi.
  - Cần sự hỗ trợ của phần cứng cho việc ánh xạ địa chỉ
  - Sử dụng trong đa số các OS đa dụng trong đó có các cơ chế swapping, paging, segmentation…

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

III. Chuyển đổi địa chỉ

Dynamic linking: là quá trình **link một module ngoài** sau khi đã tạo xong load module.

- Module ngoài: trong Windows, các module ngoài thường là các file .DLL, Trong Unix/Linux, module ngoài là các file .so.
- Load module: chương trình chính đã được biên dịch và liên kết, chứa mã thực thi và các stub. Stub là đoạn mã nhỏ được chèn vào load module để đại diện cho các hàm thuộc module ngoài. Thay vì chứa toàn bộ mã của hàm, stub chỉ lưu thông tin tham chiếu đến routine của module ngoài.
- Quá trình thực thi: khi stub được thực thi lần đầu (do process gọi routine lần đầu), stub nạp routine vào bộ nhớ, tự thay thế bằng địa chỉ của routine và routine được thực thi (stub cần sự hỗ trợ của HĐH).

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

III. Chuyển đổi địa chỉ

Ưu điểm của Dynamic linking:

- Thông thường, external module là một thư viện cung cấp các tiện ích của OS. Các chương trình thực thi có thể dùng các phiên bản khác nhau của external module mà không cần sửa đổi, biên dịch lại.
- Chia sẻ mã (code sharing): một external module chỉ cần nạp vào bộ nhớ một lần. Các tiến trình cần dùng external module này thì cùng chia sẻ đoạn mã của external module ⇒ tiết kiệm không gian nhớ và đĩa.
- Phương pháp dynamic linking cần sự hỗ trợ của OS trong việc kiểm tra xem một thủ tục nào đó có thể được chia sẻ giữa các tiến trình hay là phần mã của riêng một tiến trình (bởi vì chỉ có OS mới có quyền thực hiện việc kiểm tra này.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

III. Chuyển đổi địa chỉ

Dynamic loading:

Cơ chế: chỉ khi nào **cần được gọi đến thì một thủ tục mới được nạp vào bộ nhớ chính** ⇒ tăng độ hiệu dụng của bộ nhớ bởi vì các thủ tục không được gọi đến sẽ không chiếm chỗ trong bộ nhớ.

Rất hiệu quả trong trường hợp tồn tại khối lượng lớn mã chương trình có tần suất sử dụng thấp, không được sử dụng thường xuyên (ví dụ các thủ tục xử lý lỗi).

Hỗ trợ từ hệ điều hành

- Thông thường, user chịu trách nhiệm thiết kế và hiện thực các chương trình có dynamic loading.
- Hệ điều hành chủ yếu cung cấp một số thủ tục thư viện hỗ trợ, tạo điều kiện dễ dàng hơn cho lập trình viên.

Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

IV. Mô hình quản lý bộ nhớ

Trong chương này, mô hình quản lý bộ nhớ là một mô hình đơn giản, không có bộ nhớ ảo.

Các cơ chế quản lý bộ nhớ:

- Phân chia cố định (fixed partitioning)
- Phân chia động (dynamic partitioning)
- Phân trang đơn giản (simple paging)
- Phân đoạn đơn giản (simple segmentation)

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

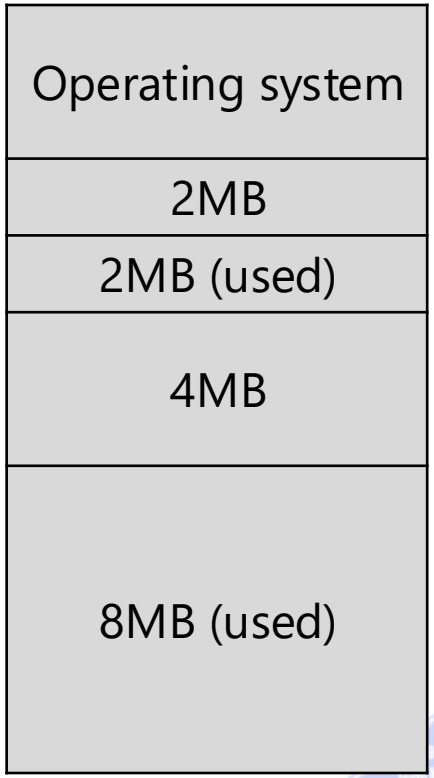
IV. Mô hình quản lý bộ nhớ

Phân mảnh:

Phân mảnh ngoại: kích thước không gian nhớ còn trống đủ để thỏa mãn một yêu cầu cấp phát, tuy nhiên không gian nhớ này không liên tục.

➔ có thể dùng cơ chế kết khối (compaction) để gom lại thành vùng nhớ liên tục.

Phân mảnh nội: kích thước vùng nhớ được cấp phát có thể hơi lớn hơn vùng nhớ yêu cầu. Hiện tượng phân mảnh nội thường xảy ra khi bộ nhớ thực được chia thành các khối kích thước cố định (fixed-sized block) và các tiến trình được cấp phát theo đơn vị khối.



Tiến trình P yêu cầu 6MB?

Tiến trình Q yêu cầu 5MB?

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

IV. Mô hình quản lý bộ nhớ

1. Phân chia cố định – fixed partitioning

Khi khởi động hệ thống, bộ nhớ chính được chia thành nhiều phần rời nhau gọi là các partition có kích thước bằng nhau hoặc khác nhau.

Tiến trình nào có kích thước nhỏ hơn hoặc bằng kích thước partition thì có thể được nạp vào partition đó.

Nếu chương trình có kích thước lớn hơn partition thì phải dùng cơ chế overlay.

➔ Gây ra hiện tượng phân mảnh nội.

Tiến trình P yêu cầu 8MB, tiến trình Q yêu cầu 5MB???

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

IV. Mô hình quản lý bộ nhớ

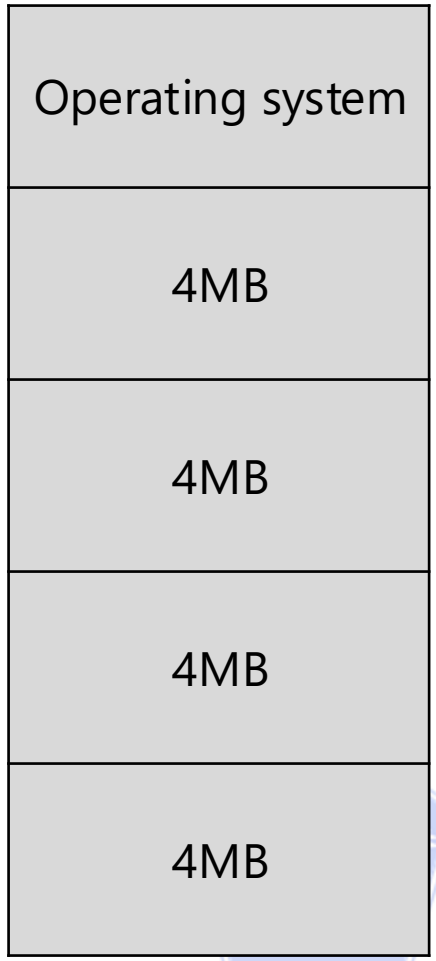
1. Phân chia cố định – fixed partitioning

Chiến lược placement

Partition có kích thước bằng nhau

Nếu còn partition trống ⇒ tiến trình mới sẽ được nạp vào partition đó.

Nếu không còn partition trống, nhưng trong đó có tiến trình đang bị blocked ⇒ swap tiến trình đó ra bộ nhớ phụ nhường chỗ cho tiến trình mới.



Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

IV. Mô hình quản lý bộ nhớ

1. Phân chia cố định – fixed partitioning

Chiến lược placement

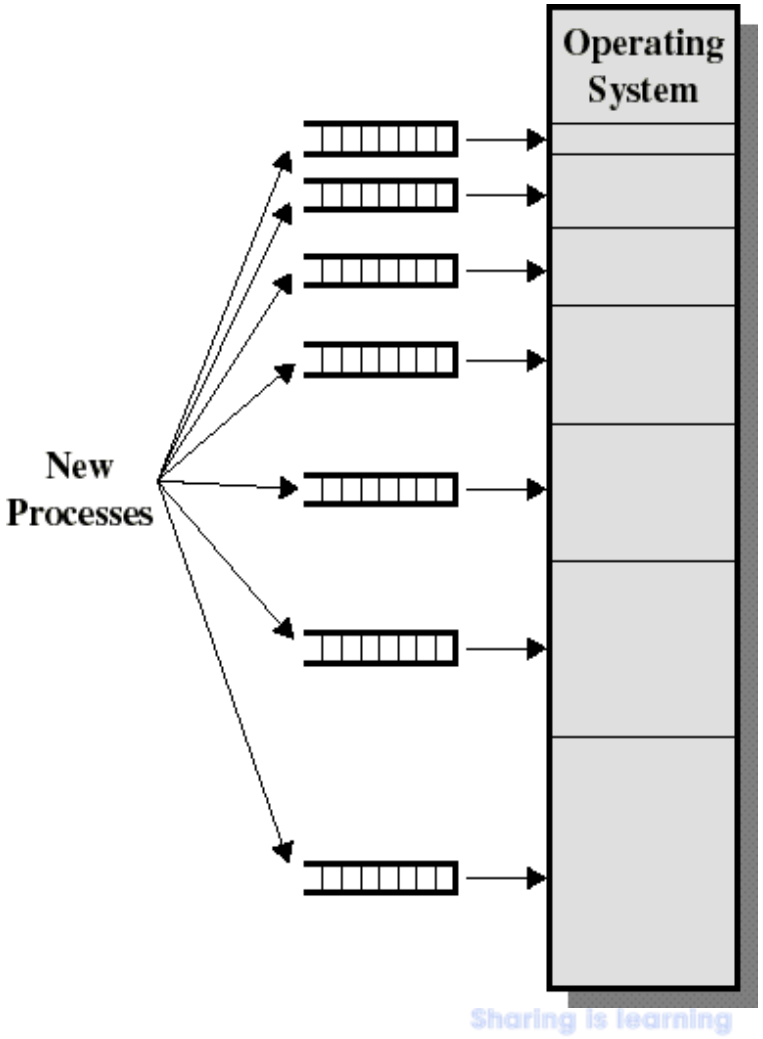
Partition có kích thước khác nhau – giải pháp 1

Gán mỗi tiến trình vào partition nhỏ nhất phù hợp với nó.

Có hàng đợi cho mỗi partition.

➔ Giảm thiểu phân mảnh nội.

Tuy nhiên: có thể có một số hàng đợi trống không (vì không có tiến trình với kích thước tương ứng) và hàng đợi dày đặc.



Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

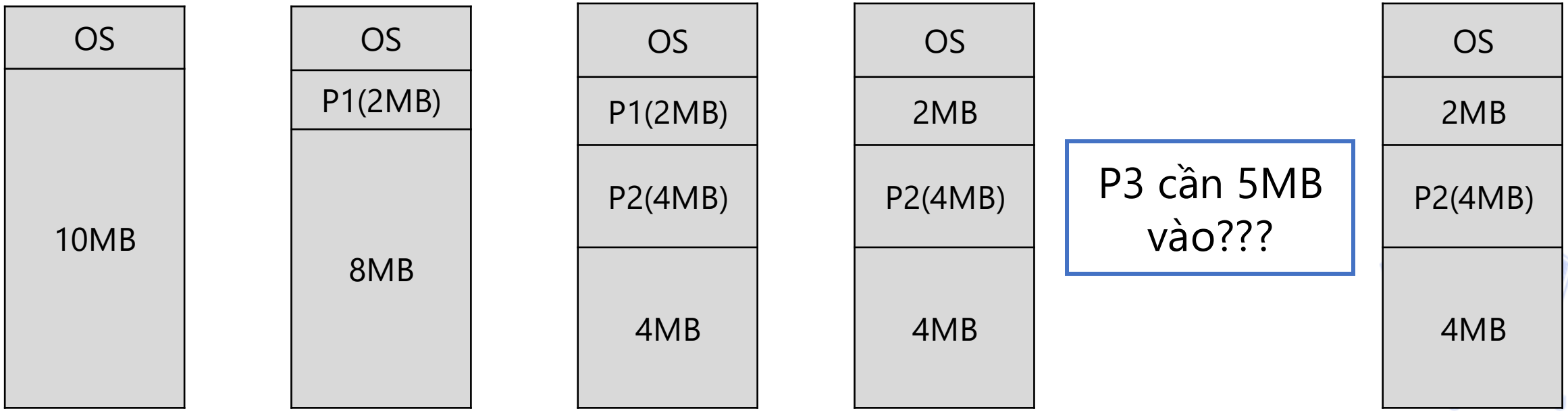
IV. Mô hình quản lý bộ nhớ

1. Phân chia động – dynamic partitioning

Số lượng partition không cố định và partition có thể có kích thước khác nhau.

Mỗi tiến trình được cấp phát chính xác dung lượng bộ nhớ cần thiết.

➔ Gây ra hiện tượng phân mảnh ngoại.



CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

IV. Mô hình quản lý bộ nhớ

1. Phân chia động – dynamic partitioning - chiến lược placement

Dùng để quyết định cấp phát khối bộ nhớ trống nào cho một tiến trình.

Mục tiêu: giảm chi phí compaction.

Các chiến lược placement

- Best-fit: chọn khối nhớ trống nhỏ nhất.
- First-fit: chọn khối nhớ trống phù hợp đầu tiên kể từ đầu bộ nhớ.
- Next-fit: chọn khối nhớ trống phù hợp đầu tiên kể từ vị trí cấp phát cuối cùng.
- Worst-fit: chọn khối nhớ trống lớn nhất.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

V. Cơ chế phân trang

- Cơ chế cấp phát bộ nhớ không liên tục
- Chia bộ nhớ vật lý thành các khối cố định gọi là các khung trang (frames). Kích thước của frame là lũy thừa của 2, từ khoảng 512 byte đến 16MB.
- Chia bộ nhớ luận lý thành các khối nhớ bằng nhau gọi là các trang (pages).

➔ Kích thước của page bằng kích thước của frame.

Thiết lập bảng phân trang (page table) để ánh xạ địa chỉ luận lý thành địa chỉ thực.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

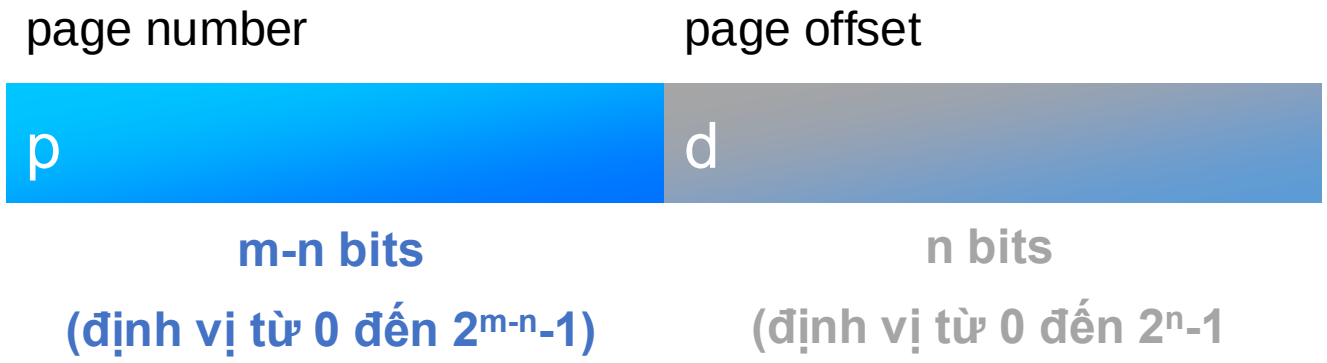
V. Cơ chế phân trang

Địa chỉ luận lý gồm có:

- Số hiệu trang (Page number) p.

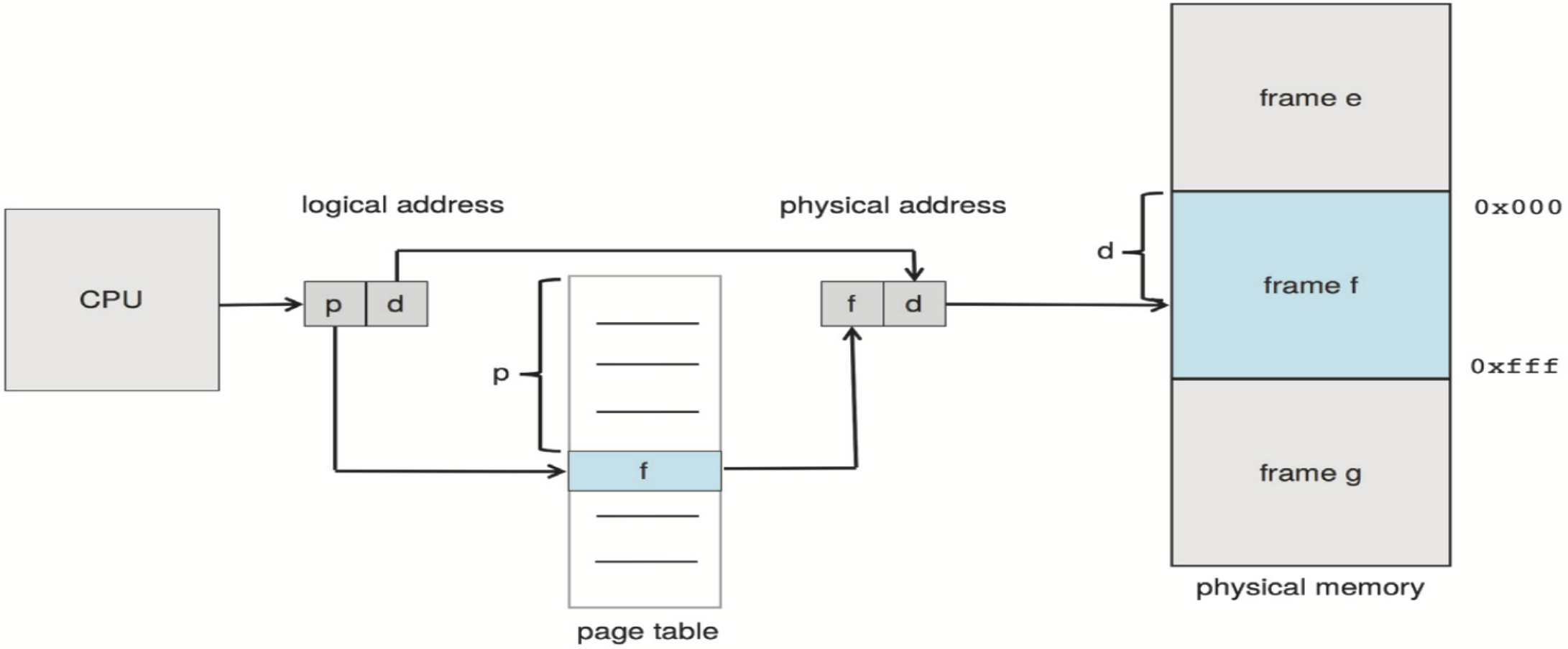
- Địa chỉ tương đối trong trang (Page offset) d.

Nếu kích thước của không gian địa chỉ ảo là 2^m, và kích thước của trang là 2^n (đơn vị là byte hay word tùy theo kiến trúc máy) thì bảng trang có 2^{m-n} mục.



CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

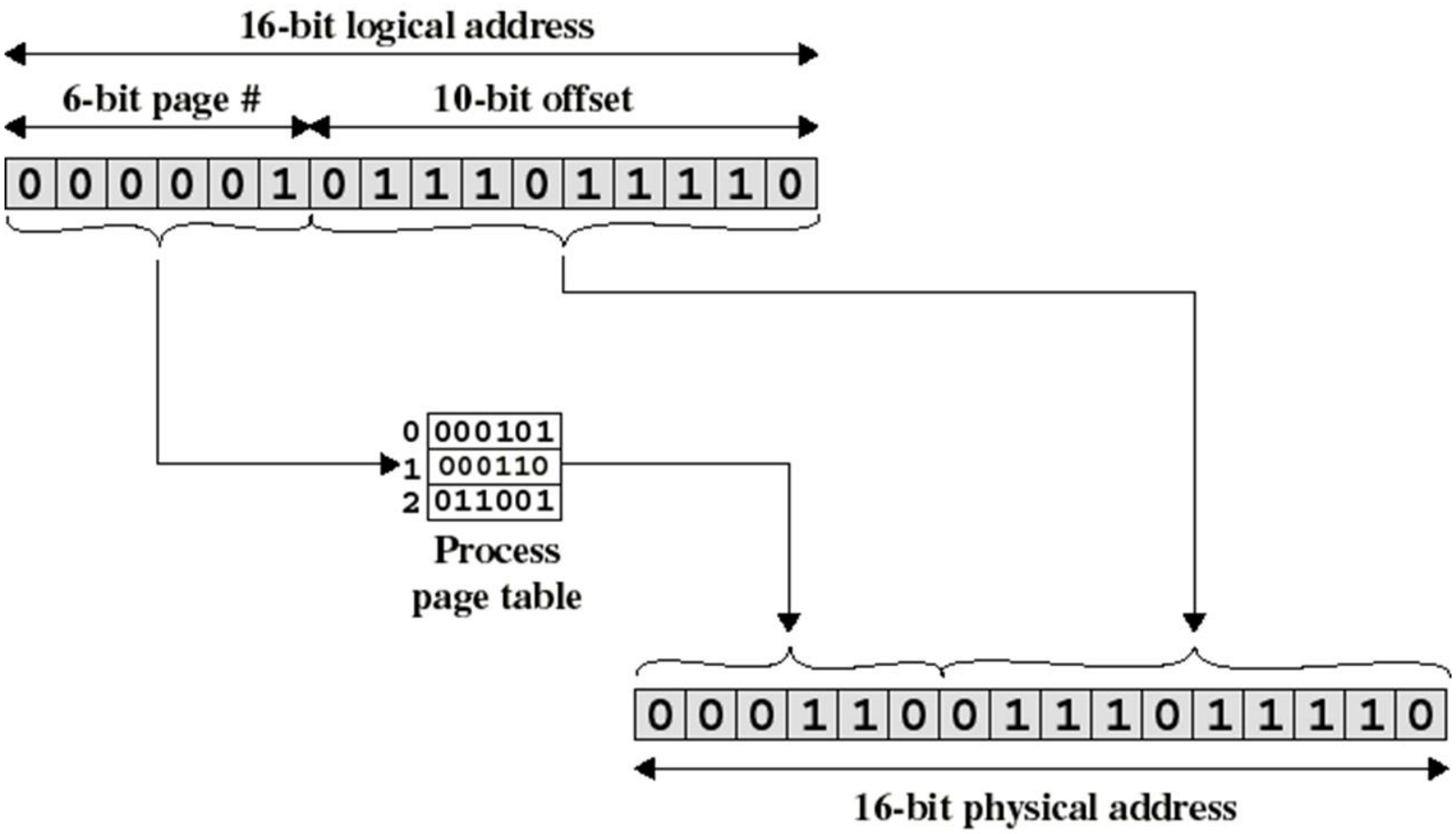
V. Cơ chế phân trang



Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

V. Cơ chế phân trang



BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

V. Cơ chế phân trang

Bảng phân trang thường được lưu giữ trong **bộ nhớ chính**:

- Mỗi tiến trình được hệ điều hành cấp một bảng phân trang.
- Thanh ghi page-table base (PTBR) trỏ đến bảng phân trang.
- Thanh ghi page-table length (PTLR) biểu thị kích thước của bảng phân trang (có thể được dùng trong cơ chế bảo vệ bộ nhớ).

Theo cơ chế cài đặt này thì một thao tác truy cập lệnh hoặc dữ liệu cần đến **2 lần truy cập bộ nhớ chính:**

- Lần 1 cho bảng trang.
- Lần 2 cho lệnh hoặc dữ liệu.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

V. Cơ chế phân trang

Thời gian truy xuất hiệu dụng (EAT):

Thời gian tìm kiếm trong TLB (associative lookup): ε

Thời gian một chu kỳ truy xuất bộ nhớ: x

Hit ratio (α): tỉ số giữa số lần chỉ số trang được tìm thấy (hit) trong TLB và số lần truy xuất khởi nguồn từ CPU.

Thời gian cần thiết để có được chỉ số frame:

Khi chỉ số trang có trong TLB (hit): ε + x

Khi chỉ số trang không có trong TLB (miss): ε + 2x

Thời gian truy xuất hiệu dụng:

$$\mathbf{EAT = (\varepsilon + x)\alpha + (\varepsilon + 2x)(1 - \alpha) = (2 - \alpha)x + \varepsilon}$$

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

V. Cơ chế phân trang

Ví dụ 1: Một không gian địa chỉ ảo có kích thước 2^32 byte, mỗi trang có kích thước 4096 byte. Hỏi bảng trang cần có bao nhiêu mục?

Số mục = 2^{32}/ 4096 = 2^{32}/ 2^{12} = 2^{20} (mục)

Ví dụ 2: Xét một không gian địa chỉ ảo có 255 trang, mỗi trang có kích thước 4096 byte được ánh xạ và bộ nhớ có 65 khung trang.

255 trang cần 8 bit, 4096 byte cần 12 bit, 65 khung cần 7 bit.

➔ p = 8, offset = 12, f = 7

Địa chỉ vật lý gồm: f + offset = 19 (bit)

Địa chỉ luận lý gồm: p + offset = 20 (bit).

Mỗi mục trong trang cần ít nhất: p = 8 (bit). **Nếu mỗi mục cần 2 byte** thì kích thước bảng phân trang là: 2 * số trang = 510 byte

Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

V. Cơ chế phân trang

Ví dụ 3: Xét một hệ thống sử dụng kỹ thuật phân trang, với bảng trang được lưu trữ trong bộ nhớ chính.

a. Nếu thời gian cho một lần truy xuất bộ nhớ thông thường là 200ns thì mất bao lâu cho một thao tác truy xuất bộ nhớ trong hệ thống này?

b. Nếu hệ thống này sử dụng TLBs với hit-radio là 70%, giả sử thời gian để tìm trong TLBs xem như bằng 0 thì thời gian truy xuất bộ nhớ trong hệ thống này là bao nhiêu?

Đáp án:

a. 200 * 2 = 400 (ns)

b. EAT = $(2 - \alpha)x + \varepsilon$ = ???

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

CÂU HỎI ÔN TẬP

1. Trong cơ chế liên kết động (dynamic linking), quá trình liên kết module ngoài được thực hiện ở giai đoạn nào?

A. Linking

B. Compile time

C. Loading

D. Executing

Đáp án: C. Loading

2. Trong cơ chế nạp động (dynamic loading), quá trình nạp module ngoài được thực hiện ở giai đoạn nào?

A. Linking

B. Compile time

C. Loading

D. Executing

Đáp án: D. Executing

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 7. QUẢN LÝ BỘ NHỚ

CÂU HỎI ÔN TẬP

3. Kỹ thuật Overlay được áp dụng khi?

A. Không còn vùng nhớ để chạy chương trình.

B. Chương trình có kích thước lớn hơn vùng nhớ.

C. Chương trình có kích thước nhỏ hơn vùng nhớ.

D. Tất cả đều sai.

Đáp án: B. Chương trình có kích thước lớn hơn vùng nhớ.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8: BỘ NHỚ ẢO

I. Tổng quan

II. Cài đặt bộ nhớ ảo

III. Phân trang theo yêu cầu

IV. Giải thuật thay trang

V. Vấn đề cấp phát Frames

VI. Vấn đề thrasing

BOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

I. Tổng quan

8.1. Định nghĩa

– **Bộ nhớ ảo (virtual memory)**: là một kỹ thuật cho phép xử lý một tiến trình không được nạp toàn bộ vào bộ nhớ vật lý.

- Ưu điểm của bộ nhớ ảo:

+ Số lượng process trong bộ nhớ nhiều hơn.

+ Một process có thể thực thi ngay cả khi kích thước của nó lớn hơn bộ nhớ thực.

+ Giảm nhẹ công việc của lập trình viên.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

II. Cài đặt bộ nhớ ảo

8.2. Các kỹ thuật cài đặt bộ nhớ ảo

– Có hai kỹ thuật:

+ Phân trang theo yêu cầu (Demand Paging)

+ Phân đoạn theo yêu cầu (Demand Segmentation)

- Để cài đặt được bộ nhớ ảo thì phần cứng memory management phải hỗ trợ page và/hoặc segmentation

- Để cài đặt kĩ thuật này thì OS phải quản lý sự di chuyển của trang/ đoạn giữa bộ nhớ chính và bộ nhớ thứ cấp.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

III.Phân trang theo yêu cầu

8.3.Cơ chế phân trang theo yêu cầu

- **Demand paging**: các trang của tiến trình chỉ được nạp vào bộ nhớ chính khi được yêu cầu.
- Khi có một tham chiếu đến một trang mà không có trong bộ nhớ chính (valid bit) thì phần cứng sẽ gây ra một ngắt (gọi là page – fault trap) kích khởi page – fault service routine (PFSR) của hệ điều hành.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

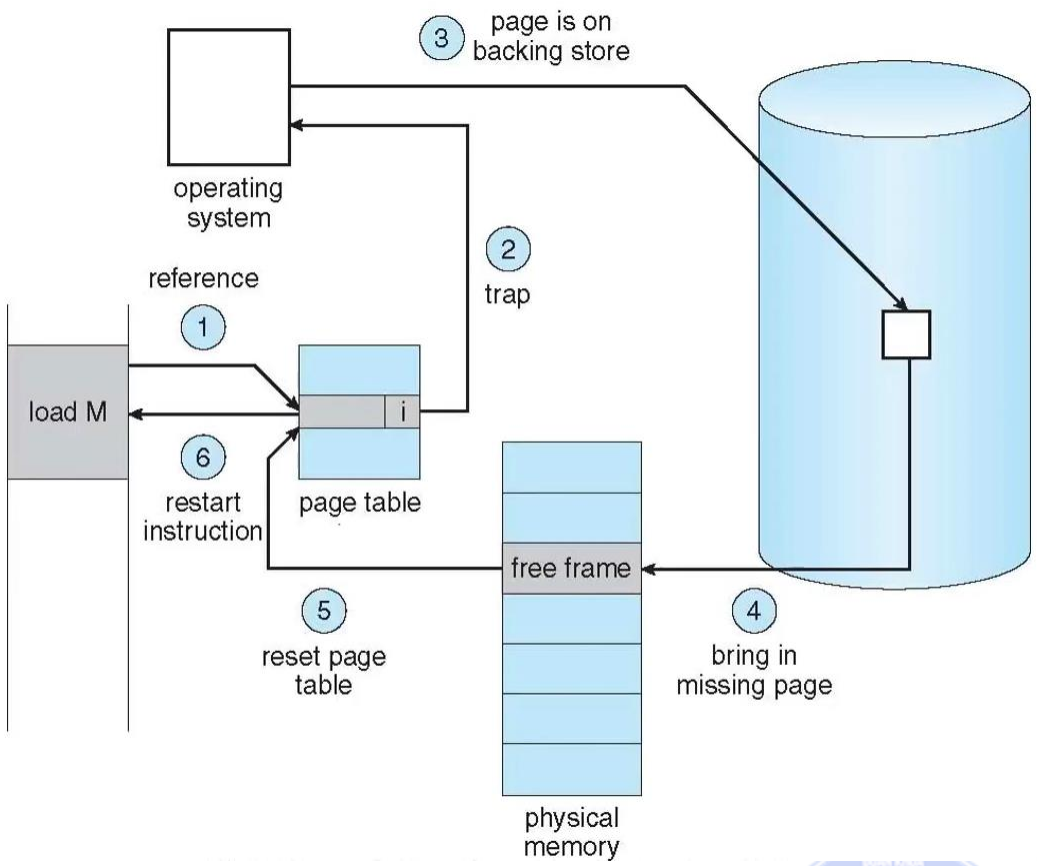
III.Phân trang theo yêu cầu

8.4.PFSR

- Chuyển process về trạng thái Blocked
- Phát ra một yêu cầu đọc đĩa để nạp trang được tham chiếu vào một frame trống; trong khi đợi I/O, một process khác được cấp CPU để thực thi
- Sau khi I/O hoàn tất, đĩa gây ra một ngắt đến hệ điều hành; PFSR cập nhật page table và chuyển process về trạng thái ready.

Nếu không tìm được frame trống:

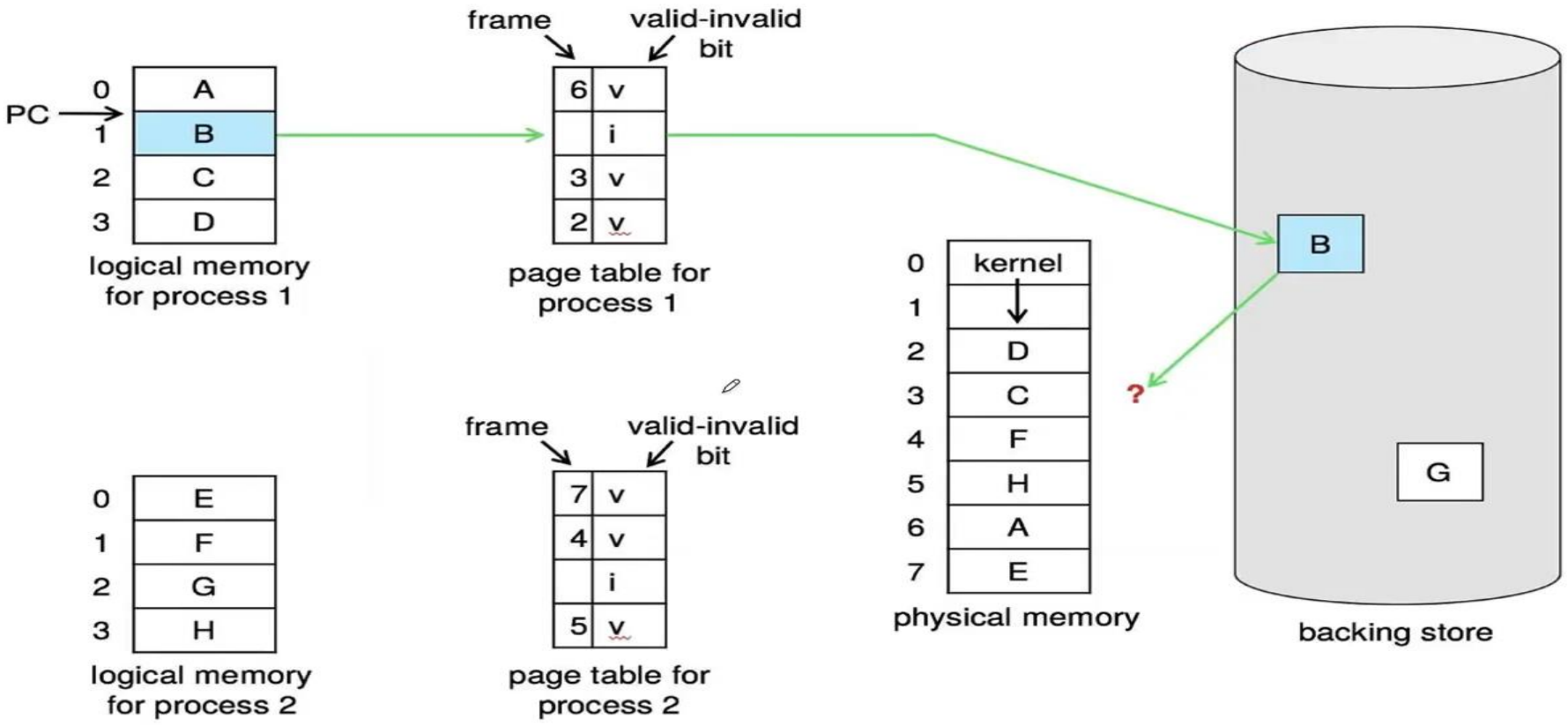
- Dùng giải thuật thay trang chọn một trang hi sinh (victim page)
- Ghi victim page lên đĩa



CHƯƠNG 8. BỘ NHỚ ẢO

III.Phân trang theo yêu cầu

8.5.Khi cần thay thế trang



CHƯƠNG 8. BỘ NHỚ ẢO

III.Phân trang theo yêu cầu

8.5. Khi cần thay thế trang

Bước 2 của PFSR giả sử phải thay trang vì không tìm được frame trống, PFSR được bổ sung như sau:

- Xác định vị trí trên đĩa của trang đang cần
- Tìm một frame trống:

→Nếu có frame trống thì dùng nó

→ Nếu không có frame trống thì dùng một giải thuật thay trang để chọn một trang hy sinh (victim page)

→ Ghi victim page lên đĩa; cập nhật page table và frame table tương ứng

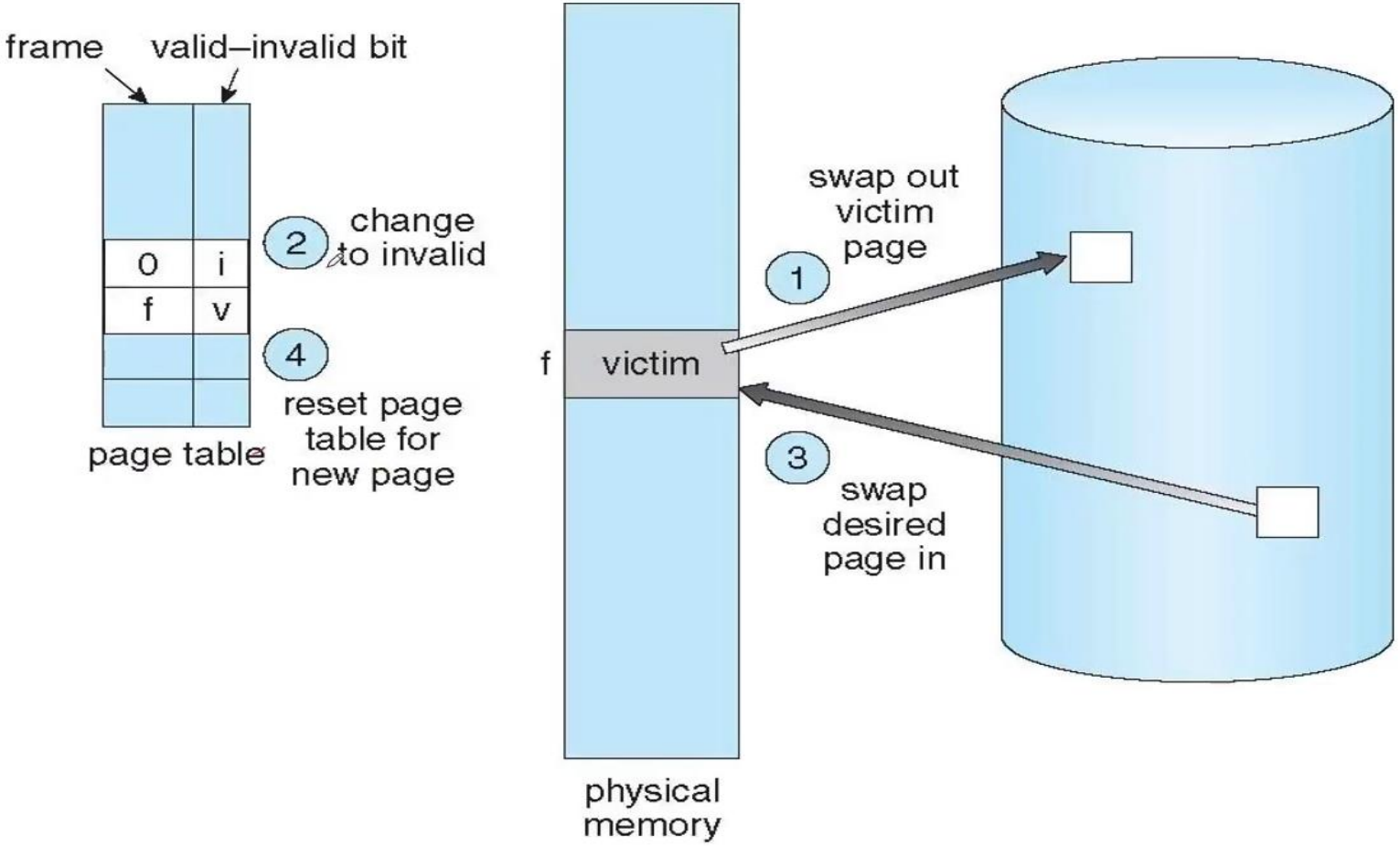
- Đọc trang đang cần vào frame trống (đã có được từ bước 2); cập nhật page table và frame table tương ứng.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

III.Phân trang theo yêu cầu

8.5. Khi cần thay thế trang



CHƯƠNG 8. BỘ NHỚ ẢO

IV. Giải thuật thay trang

8.6. Các giải thuật thay thế trang

- Các giải thuật thay thế trang:

+ Giải thuật thay trang **FIFO**

+ Giải thuật thay trang **OPT**

+ Giải thuật thay trang **LRU**

- Các dữ liệu cần ban đầu:

+ Số khung trang

+ Tình trạng ban đầu

+ Chuỗi tham chiếu

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

IV. Giải thuật thay trang

8.7. Giải thuật FIFO

Ví dụ: cho chuỗi truy xuất bộ nhớ: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

*FIFO (first in, first out): vào trước ra trước

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
	0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
		1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
*	*	*	*		*	*	*	*	*	*			*	*			*	*	*

→ 15 lỗi trang

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

IV. Giải thuật thay trang

8.8. Nghịch lý Belady

Sử dụng 3 khung trang

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4
*	*	*	*	*	*	*			*	*	

→ 9 lỗi trang

Sử dụng 4 khung trang

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3
*	*	*	*			*	*	*	*	*	*

→ 10 lỗi trang

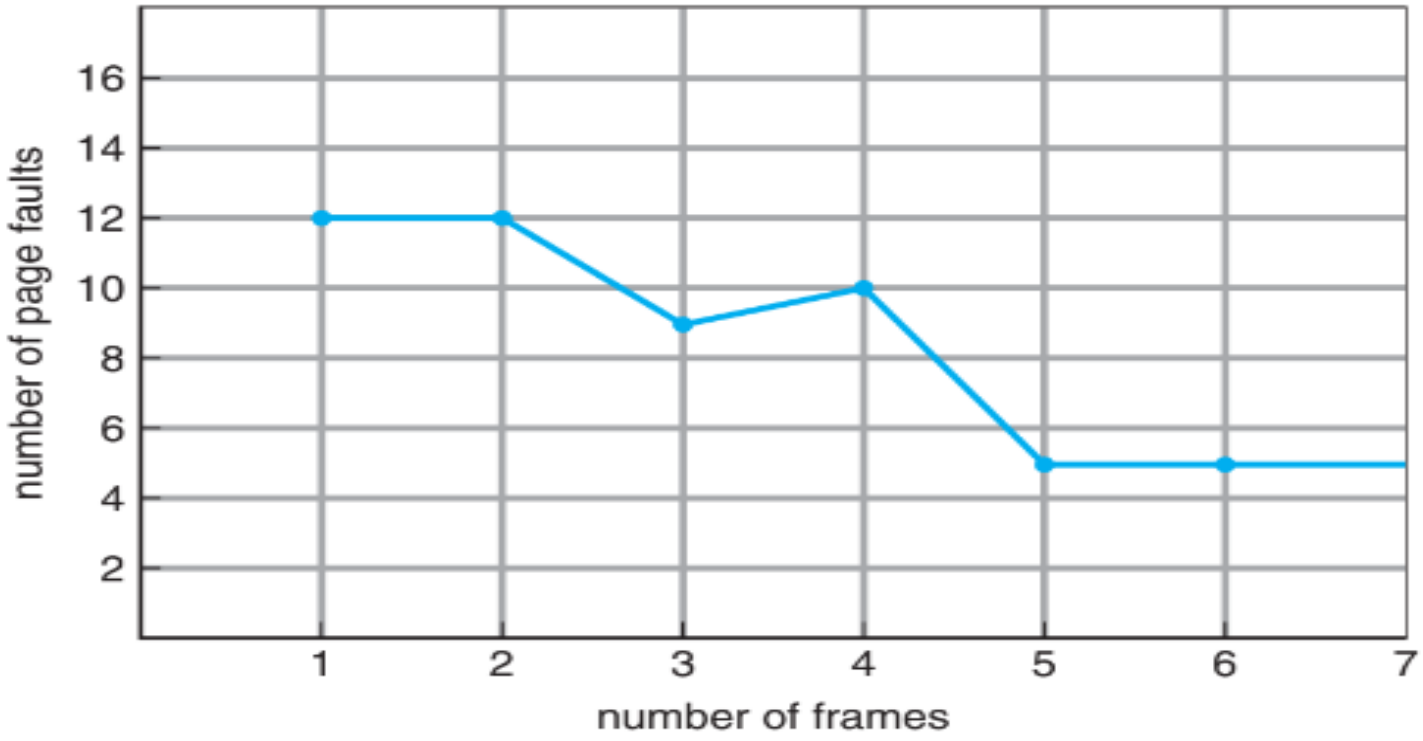
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

IV. Giải thuật thay trang

8.8. Nghịch lý Belady

– Nghịch lý Belady: số page fault tăng mặc dù quá trình đã được cấp nhiều frame hơn.



BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

IV. Giải thuật thay trang

8.9. Giải thuật OPT

- Giải thuật thay trang OPT thay thế trang nhớ sẽ được tham chiếu trễ nhất trong tương lai ⇒ cần phải biết trước các trang sẽ được tham chiếu trong tương lai

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
	0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
		1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
*	*	*	*		*		*			*			*				*		

→ 9 lỗi trang

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

IV. Giải thuật thay trang

8.10. Giải thuật LRU

- Mỗi trang được ghi nhận (trong bảng phân trang) thời điểm được tham chiếu ⇒ trang LRU là trang nhớ có thời điểm tham chiếu nhỏ nhất (OS tốn chi phí tìm kiếm trang nhớ LRU này mỗi khi có page fault).

- Do vậy, LRU cần sự hỗ trợ của phần cứng và chi phí cho việc tìm kiếm. Ít CPU cung cấp đủ sự hỗ trợ phần cứng cho giải thuật LRU.

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
*	*	*	*		*		*	*	*	*			*		*		*		

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

V. Vấn đề cấp phát frames

8.11. Số lượng frame cấp cho process

- **OS phải quyết định cấp cho mỗi tiến trình bao nhiêu frame.**

+ Cấp ít frame ⇒ nhiều page fault

+ Cấp nhiều frame ⇒ giảm mức độ multiprogramming

- **Chiến lược cấp phát tĩnh (fixed-allocation)**

+ Số frame cấp cho mỗi process không đổi, được xác định vào thời điểm loading và có thể tùy thuộc vào từng ứng dụng (kích thước của nó).

- **Chiến lược cấp phát động (variable-allocation)**

+ Số frame cấp cho mỗi process có thể thay đổi khi nó chạy:

→ Nếu tỷ lệ page – fault cao ⇒ cấp thêm frame

→ Nếu tỷ lệ page – fault thấp ⇒ giảm bớt frame

- **OS** phải mất chi phí để ước định các process

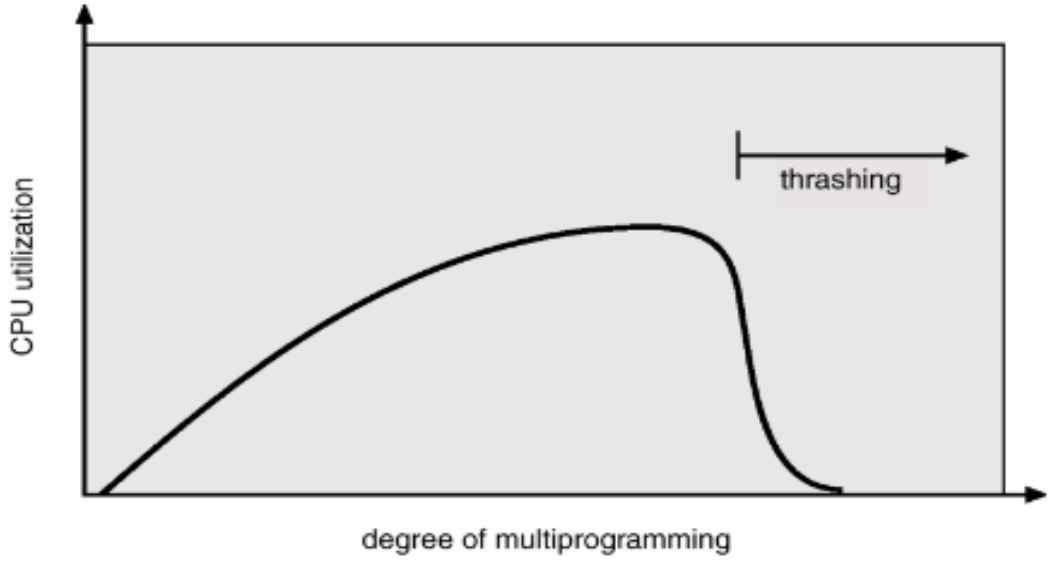
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

VI. Vấn đề thrashing

8.14. Trì trệ trên toàn bộ hệ thống

- Nếu một process không có đủ số frame cần thiết thì tỉ số page faults/sec rất cao.
- Thrasing: hiện tượng các trang nhớ của một process bị hoán chuyển vào/ra liên tục.



BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

VI. Vấn đề thrashing

8.15. Mô hình cục bộ

- Bao nhiêu frame là đủ cho một process thực thi hiệu quả?

- Nguyên lý locality (locality principle):

+ Locality là tập các trang được tham chiếu gần nhau.

+ Một process gồm nhiều locality và trong quá trình thực thi, process sẽ chuyển từ locality này sang locality khác.

- Hiện tượng thrashing xảy ra khi kích thước của tất cả các locality trên tất cả tiến trình lớn hơn kích thước bộ nhớ.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

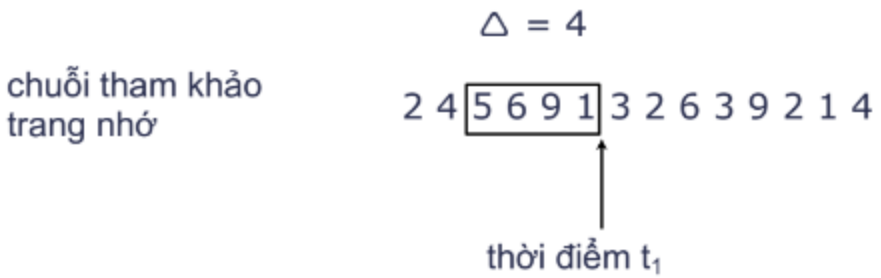
VI. Vấn đề thrashing

8.16. Giải pháp tập làm việc

- Được thiết kế dựa trên nguyên lý locality
- Xác định xem tiến trình thực sự sử dụng bao nhiêu frame
- Định nghĩa:

+ WS(t) – các tham chiếu trang nhớ của tiến trình gần đây nhất cần được quan sát.

+ Δ – khoảng thời gian tham chiếu.

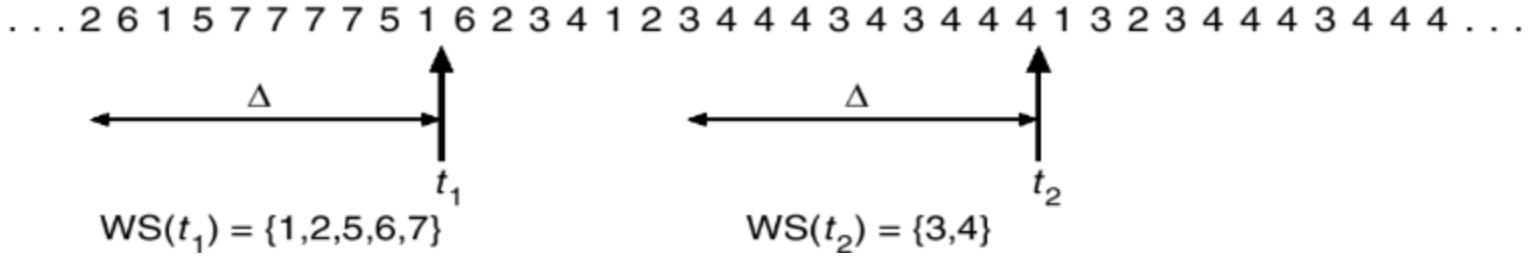


CHƯƠNG 8. BỘ NHỚ ẢO

VI. Vấn đề thrashing

8.16. Giải pháp tập làm việc

- Định nghĩa: working set của tiến trình Pi, ký hiệu Wsi, là tập gồm Δ các trang được sử dụng gần đây nhất.
- Ví dụ: Δ = 10 và tham khảo trang



- Nhận xét: Δ quá nhỏ ⇒ không bao phủ toàn bộ locality

  Δ quá lớn ⇒ bao phủ nhiều locality khác nhau

  Δ vô cực ⇒ bao gồm tất cả các trang được sử dụng
- Dùng working set của một tiến trình để xấp xỉ locality của nó

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

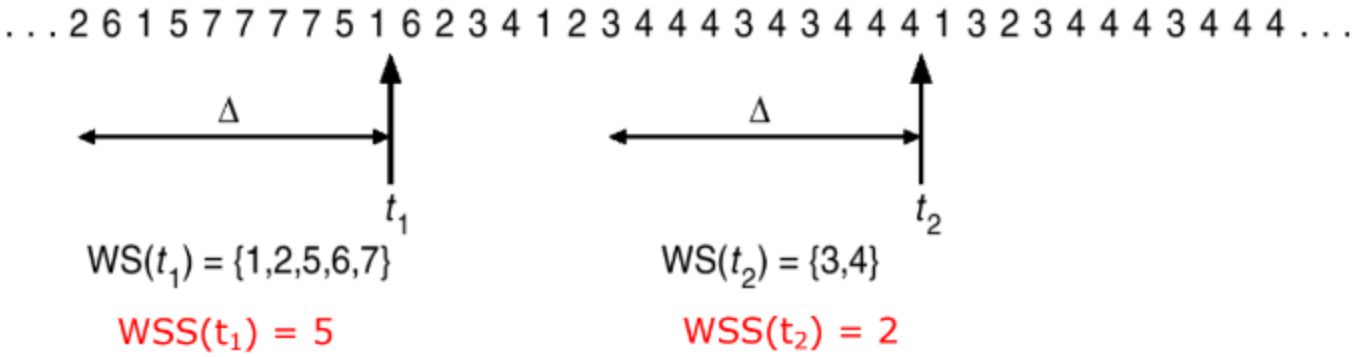
VI. Vấn đề thrashing

8.16. Giải pháp tập làm việc

- Định nghĩa: WSSi là kích thuớc của working set của Pi:

+ WSSi = số lượng các trang trong Wsi

+ Ví dụ: Δ = 10 và chuỗi tham khảo trang



CHƯƠNG 8. BỘ NHỚ ẢO

VI. Vấn đề thrashing

8.16. Giải pháp tập làm việc

- Đặt D bằng tổng tất cả các working – set size của mọi tiến trình trong hệ thống.

+ Nhận xét: nếu D > m (số frame của hệ thống) ⇒ sẽ xảy ra thrashing

- Giải pháp working set:

+ Khi khởi tạo một tiến trình: cung cấp cho quá trình số lượng frame thỏa mãn working set size của nó.

+ Nếu D > m ⇒ tạm dừng một trong các tiến trình.

→ Các trang của tiến trình được chuyển ra đĩa cứng và các frame của nó được thu hồi.

- WS loại trừ được tình trạng trì trệ mà vẫn đảm bảo mức độ đa chương.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

Một số câu hỏi về bộ nhớ ảo

Câu 1 (CK2 năm 2023-2024): Chọn phát biểu sai về việc cấp phát khung trang

A. Nếu hệ điều hành cấp phát ít khung trang cho tiến trình thì sẽ làm xuất hiện nhiều lỗi trang.

B. Nếu hệ điều hành cấp phát nhiều khung trang cho tiến trình thì sẽ làm giảm mức độ đa chương.

C. Số lượng khung trang được cấp cho mỗi tiến trình sẽ không thay Đổi nếu sử dụng chiến lược cấp phát tĩnh.

D. Khi sử dụng chiến lược cấp phát động, nếu tỉ lệ lỗi trang cao, hệ điều hành sẽ giảm bớt số lượng khung trang được cấp.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

Một số câu hỏi về bộ nhớ ảo

Câu 1 (CK2 năm 2023-2024): Chọn phát biểu SAI về việc cấp phát khung trang

A. Nếu hệ điều hành cấp phát ít khung trang cho tiến trình thì sẽ làm xuất hiện nhiều lỗi trang.

B. Nếu hệ điều hành cấp phát nhiều khung trang cho tiến trình thì sẽ làm giảm mức độ đa chương.

C. Số lượng khung trang được cấp cho mỗi tiến trình sẽ không thay Đổi nếu sử dụng chiến lược cấp phát tĩnh.

D. Khi sử dụng chiến lược cấp phát động, nếu tỉ lệ lỗi trang cao, hệ điều hành sẽ giảm bớt số lượng khung trang được cấp.

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

Một số câu hỏi về bộ nhớ ảo

Câu 2 (CK2 năm 2023-2024): Trả lời ngắn gọn câu hỏi sau bằng tiếng Anh

Kỹ thuật nào cho phép xử lý một tiến trình không được nạp toàn bộ vào bộ nhớ vật lý?

BAN HỌC TẬP
Sharing is learning

CHƯƠNG 8. BỘ NHỚ ẢO

Một số câu hỏi về bộ nhớ ảo

Câu 2 (CK2 năm 2023-2024): Trả lời ngắn gọn câu hỏi sau bằng tiếng Anh

Kỹ thuật nào cho phép xử lý một tiến trình không được nạp toàn bộ vào bộ nhớ vật lý?

Trả lời: Demand Paging

BAN HỌC TẬP
Sharing is learning

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
HỌC THUẬT

- Training
- Giải đáp
- Chia sẻ

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
TRUYỀN THÔNG

- Design ấn phẩm
- Viết content
- Chụp ảnh

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
TIẾNG ANH

- Instagram
- TikTok
- Dịch thuật
- Thi thử

BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

TRAINING GIỮA KỲ HỌC KỲ II NĂM HỌC 2023 – 2024

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

HẾT

CẢM ƠN CÁC BẠN ĐÃ THEO DÕI
CHÚC CÁC BẠN CÓ KẾT QUẢ THI THẬT TỐT!

BAN HỌC TẬP

Khoa Công nghệ Phần mềm
Trường Đại học Công nghệ Thông tin
Đại học Quốc gia thành phố Hồ Chí Minh

CONTACT

bht.cnpm.uit@gmail.com
fb.com/bhtcnpm
fb.com/groups/bht.cnpm.uit

TEAM TIẾNG ANH

english.with.bht@gmail.com
creative.owl.se
english.with.bht